



Security Implementation Specification

(v1.2)

Theft Mitigation · Attack-Vector Countermeasures · DAG Security · Verification Methodology

Version	v1.3 — Baseline (v1.0) + DAG + Token Security Standards + Quantum Security Categories
Classification	Internal Technical Specification for architecture, protocol engineering, validator operations, audit, and red-team execution
Document Scope	24 attack categories (14 baseline + 1 token-authenticity + 3 DAG-specific + 6 quantum-specific) · 50+ attack methods · 15+ protocol defenses · 11 SynQ language guarantees · Quantum threat timeline and emergency governance protocol
Layers Covered	L1 Protocol (Consensus) · SynQ Language (Compiler/VM) · Wallet Architecture (Three-Plane Model) · Cluster-Local DAG Sublayer · Quantum Threat Surface (Cross-Chain and Protocol)
Quantum Threat Status	ACTIVE ENGINEERING CONSTRAINT — NOT A FUTURE CONCERN 6,100 coherent atomic qubits demonstrated (Manetsch et al. 2025). ~10,000 required for ECC-256 break. The gap is engineering, not fundamental science.
Status	Working specification — implementation details are normative; test vectors are illustrative
v1.3 Changes	Quantum threat timeline added; CRQC attack taxonomy added; six new attack categories (19–24); quantum emergency governance protocol; misalignment corrections to executive summary, Category 8, and Category 12; all quantum-as-future-concern framing removed.

SECURITY CRITICAL — INTERNAL USE ONLY — CONFIDENTIAL

Prepared for the Synergy Network validator-cluster rollout. This update closes the exact security gap created when a high-throughput DAG data plane is introduced beneath a deterministic BFT settlement layer. The document does not soften any existing Synergy protections; it hardens them for the new attack surface.

Executive Summary

This document is the authoritative technical specification for every theft-mitigation measure that Synergy Network will implement at the protocol, language, and wallet layers. It is organized around twenty-four major attack categories — fourteen from the baseline specification, one token-authenticity category added in v1.1, three DAG-specific categories introduced in v1.1, and six quantum-specific categories added in this revision (v1.3) — and expands each countermeasure into a concrete implementation plan with step-by-step engineering tasks and verification methodology.

The central thesis is that most cryptocurrency theft does not exploit bugs in individual contracts — it exploits architectural assumptions inherited from Ethereum's original design. Unlimited approvals exist because ERC-20 was designed before DeFi. Blind signing persists because transaction formats predate complex dApp interactions. MEV is structural to public mempools. Reentrancy is possible because the EVM allows arbitrary external calls mid-execution. The DAG layer introduces new surface — dissemination attacks, ordering manipulation, replay across epochs, and control-plane abuse — that must be treated as first-class security problems, not operational footnotes.

Quantum computing introduces a category of threat that is categorically different from all of the above: it does not exploit implementation errors, it attacks the mathematical foundations of classical cryptography directly. Both papers evaluated in March 2026 (Cain et al., arXiv:2603.28627; Babbush et al., Google Quantum AI / Ethereum Foundation) establish that cryptographically-relevant quantum computers (CRQCs) capable of breaking ECC-256 are within engineering reach — not theoretical future reach. With 6,100 coherent atomic qubits already demonstrated and approximately 10,000–26,000 required to break ECC-256, the remaining gap is an engineering milestone, not a fundamental science barrier. The correct framing is: quantum migration is an active engineering constraint, and it must be treated as such in every layer of this specification.

Synergy Network addresses all of this at the root. Rather than layering countermeasures on top of an insecure base, the protocol is designed so that insecure patterns are structurally impossible. Synergy's cryptographic algorithm selection — ML-DSA-65/87 for all signing, ML-KEM-768 for key encapsulation, SLH-DSA for long-lived credentials, and STARK-based ZK proofs — is confirmed correct by both quantum papers. The protocol uses PQC from genesis. There is no classical key fallback and no safe interim period.

Architecture Context: Three Defense Layers

Every countermeasure in this document operates at one or more of three distinct defense layers within the Synergy stack. Failures in one layer must not cascade to others — this principle of fail-closed, plane-isolated containment is the architectural thread that runs through the entire system.

Layer 1 — L1 Protocol (Consensus Layer)

The consensus layer enforces rules that cannot be overridden by any application, wallet, or smart contract. Defenses implemented at this layer are universally and automatically applied: time-delayed transfers, encrypted mempools, native naming systems, approval expiry enforcement, rate limiting, and cross-chain proof verification all live here. If an action violates a consensus-layer rule, it is rejected unconditionally before execution.

Layer 2 — SynQ Language (Compiler and VM)

SynQ is a purpose-built smart-contract language whose type system and compiler eliminate entire vulnerability classes by making them inexpressible. Linear asset types make reentrancy structurally impossible. Bounded loops prevent gas-exhaustion attacks. The absence of raw delegatcall eliminates storage-collision exploits. These are compile-time guarantees, not runtime checks. A contract that would be vulnerable cannot compile.

Layer 3 — Wallet Architecture (Three-Plane Model)

The Synergy Wallet is a protocol component, not a peripheral application. It enforces the Local Policy Engine as a hard pre-signature gate, separates Identity, Routing, and Execution into isolated planes, and requires guardian-mediated recovery with time-locks. Compromise of the wallet does not imply compromise of identity or authority because the architecture explicitly decouples the two.

KEY PRINCIPLE: Defenses are additive across layers. An attack that bypasses a wallet-layer control still faces protocol-layer enforcement. An attack that bypasses the protocol layer is blocked by the SynQ type system. No single layer is the last line of defense.

Quantum Algorithm Threat Decomposition

This section establishes the foundational quantum threat model for Synergy Network. It is a prerequisite for the six quantum-specific attack categories (19–24) that follow.

Shor's Algorithm vs. Grover's Algorithm

Quantum computing presents two distinct algorithmic threats to cryptographic systems. These are not equivalent and must not be conflated.

Algorithm	What It Breaks	Speedup	Synergy Impact
Shor's Algorithm	ECDSA / secp256k1 / Ed25519 / ECDH / X25519 / RSA — any discrete-log or integer-factoring problem	Polynomial time — exponentially faster than classical. Completely breaks 256-bit ECC.	Primary threat. All cross-chain ECDSA keys are Shor-vulnerable. Synergy's own keys (ML-DSA) are Shor-resistant.
Grover's Algorithm	Hash functions (SHA3-256, BLAKE3). Provides quadratic speedup on brute-force search.	Quadratic — halves effective bit security. SHA3-256 → ~128-bit post-quantum security. Grover-based PoW attacks are not practically relevant in the next several decades (Paper 2).	Acceptable. SHA3-256 nullifiers and commitments retain 128-bit post-quantum security. No design change required. STARK proofs using hash-based commitments are Grover-resistant.

Normative statement: Synergy's ML-DSA signing is Shor-resistant. Synergy's SHA3-256 hash usage in nullifiers and commitments is Grover-resistant at the 128-bit security level. These properties are protocol invariants and must be preserved in all future upgrades.

CRQC Hardware Taxonomy: Fast-Clock vs. Slow-Clock

Not all CRQCs present the same threat profile. The attack types that are possible depend on the hardware's clock speed — specifically, how fast it can complete a single Shor's algorithm run against ECC-256. This distinction is critical for threat modeling and must be tracked as hardware advances.

CRQC Type	Hardware	ECC-256 Break Time	Attack Types Possible	Blockchain Impact
Fast-clock CRQC	Superconducting, photonic, silicon spin	~9 minutes (Paper 2, verified via ZK proof)	On-spend attack + At-rest attack	Can steal funds from active transactions in mempool within block time. Most chains are vulnerable. Synergy's BFT finality + encrypted mempool provides defense.
Slow-clock CRQC	Neutral atoms, ion traps	~10 days with 26,000 qubits (Paper 1). 6,100 qubits already demonstrated.	At-rest attack only (initially)	Can derive private keys for long-lived exposed keys: validators, governance, treasury, staking. Cross-chain ECDSA keys are at-rest targets.

KEY THREAT MODEL SUMMARY:

Synergy's encrypted mempool defends against fast-clock on-spend attacks. Synergy's ML-DSA validator keys eliminate the consensus attack surface. The primary residual risk is at-rest attack against cross-chain ECDSA keys. This is the threat surface addressed by Categories 20 and 23.

Attack Category 1: ERC-20 Approval and Allowance Exploits

1.1 Threat Description

The ERC-20 `approve()` / `transferFrom()` model was designed for a world where one party occasionally delegates spending authority to another. In practice, dApps request unlimited ($2^{256}-1$) approvals for convenience and users grant them without understanding the long-lived attack surface this creates. An attacker who compromises a contract — through a frontend injection, a supply-chain attack, or a protocol upgrade — can call `transferFrom()` to drain every wallet with an outstanding approval, even if those wallets have not been touched again.

The BadgerDAO incident (\$120M, December 2021) is the canonical case: attackers compromised a Cloudflare API key, silently harvested `increaseAllowance()` approvals over 12 days, then drained all 500 affected wallets in a single operation. No private key was ever compromised. The approval mechanism itself was the vulnerability.

1.2 Synergy Implementation: Mandatory Scoped Approvals

SynQ eliminates unlimited approvals entirely. Every approval is a protocol-enforced struct — it cannot be expressed in any other form — and the struct requires five mandatory fields. The protocol consensus layer rejects any approval message that omits any of these fields.

Field	<code>spender</code> (address), <code>amount_cap</code> (u256), <code>expiry_timestamp</code> (u64), <code>function_selector</code> (bytes4), <code>parameter_constraints</code> (policy object)
No Expiry	Rejected at consensus — transactions containing approval structs without <code>expiry_timestamp</code> are invalid and will never be included in a block
<code>revokeAll()</code> type	A native transaction type (not a contract function) that atomically clears every outstanding approval from a given account in a single consensus operation
Max Approval Cap	Protocol-configurable upper bound on <code>amount_cap</code> per approval type; governance can set per-token or per-protocol global caps
Approval Audit Log	All approval creations, uses, and expirations are emitted as indexed consensus-level events, permanently queryable via the <code>rpc_getApprovalHistory</code> endpoint

1.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Define ApprovalStruct schema in consensus	Specify the exact binary encoding of the 5-field approval struct in the Synergy transaction format specification. This becomes part of the consensus protocol, not a contract ABI. The schema must be versioned so future fields can be added without breaking existing approvals.	Submit a transaction containing an approval struct with expiry_timestamp omitted. The node must reject with INVALID_APPROVAL_STRUCT before mempool admission.
2	Implement expiry enforcement in block validation	Validators must check expiry_timestamp against the current block timestamp when processing any transferFrom-equivalent call. Expired approvals must fail with APPROVAL_EXPIRED. This check runs before any SynQ contract code executes.	Create an approval with expiry 30 seconds in the future. Wait 31 seconds. Attempt to use it. Confirm consensus-layer rejection with APPROVAL_EXPIRED without any contract invocation.
3	Build revokeAll() as a native transaction type	Add a new transaction type REVOKE_ALL to the transaction type enum. The state transition function must atomically set all approval expiry_timestamps for the sender account to block.timestamp - 1. This is a consensus-layer operation with O(n) complexity bounded by a per-block approval-count limit.	Issue a revokeAll() with 50 outstanding approvals. Verify in the same block that all 50 appear in the approval audit log as expired, and that subsequent transfer attempts using any of them fail.
4	Implement approval audit log indexing	The consensus state machine must emit an ApprovalEvent (created/used/expired/revoked) for every approval state change. These events must be indexed by (owner, spender, token) and queryable via the RPC approval-history endpoint.	Query rpc_getApprovalHistory for a test account. Confirm every approval creation, partial use, and expiry appears with correct timestamps and amounts.
5	Integrate with Local Policy Engine	The wallet's Local Policy Engine must evaluate all approval requests before displaying them. Policy rules must default to: reject amount_cap > configurable threshold, reject expiry > 30 days, warn if function_selector grants broad transfer authority.	Attempt to create an approval with amount_cap = 2^256-1 via the reference wallet. Confirm the policy engine blocks it before presenting any signing UI.

1.4 Verification Tests

TEST SUITE 1A: Approval Struct Validation — Submit malformed approval structs (missing each field individually, then in combination). All must be rejected by consensus. Zero malformed structs must enter the mempool.
TEST SUITE 1B: Expiry Enforcement — Create approvals at epoch T with various expiry offsets. At T+offset+1, attempt to use each. All expired approvals must fail at the consensus layer. Target false-positive rate: zero.
TEST SUITE 1C: revokeAll() Atomicity — Confirm that revokeAll() revokes ALL approvals in a single block with no partial success states. Test with concurrent transferFrom() calls racing against revokeAll() in the same block.

Attack Category 2: Permit and Permit2 Signature Phishing

2.1 Threat Description

EIP-2612 `permit()` and Uniswap's Permit2 enable gasless, off-chain signature-based approvals. The attack flow is devastatingly simple: a drainer website queries the victim's balances, generates permit messages, presents them as benign "sign this message" requests, captures the signatures, and immediately submits them on-chain. In H1 2024, 260,000 victims lost \$314M to permit-based phishing — already exceeding the entire previous year. Permit and Permit2 signatures accounted for 38% of cases exceeding \$1M.

2.2 Synergy Implementation: Protocol-Native Intent System

Synergy eliminates the off-chain signature attack vector at the protocol level by replacing it with a mandatory on-chain intent submission model. No approval-equivalent operation can be authorized through an off-chain message alone.

Core Change	All approval-equivalent operations must be submitted as on-chain transactions. Off-chain messages are scoped by the protocol to read-only operations only and cannot authorize any token transfer or state mutation.
<code>simulate()</code> opcode	A native <code>simulate()</code> opcode is available in the SynQ VM. Every transaction must pass simulation before it can be submitted. The simulation result is a human-readable asset flow description that must be presented to the user before signing.
Off-chain signature scope	Off-chain EIP-712-style signatures are restricted at the protocol level to operations flagged as <code>@readonly</code> in the SynQ function declaration. Any attempt to use an off-chain signature to authorize a state-mutating operation is rejected by the consensus layer.
No gasless drain path	Because permits require on-chain submission, the attacker must pay gas. This does not eliminate the threat but eliminates the "zero trace until too late" property that makes permit phishing uniquely dan

2.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement <code>simulate()</code> as a native opcode in SynQ VM	The opcode takes a transaction object and returns a structured <code>SimulationResult</code> containing: (a) exact asset movements, (b) authorization objects created or consumed, (c) approvals granted, and (d) any detected simulation-vs-execution divergence flags. The result must be deterministic.	Write 20 transactions with known expected asset flows. Run <code>simulate()</code> on each. Confirm all 20 produce correct structured results. Mutate each transaction's calldata slightly and confirm <code>simulate()</code> reflects the mutation.
2	Enforce on-chain submission requirement for state-mutating operations	Add a SynQ function annotation: <code>@readonly</code> for view functions and <code>@mutating</code> for state-changing functions. The consensus layer must reject any transaction where an off-chain signature is presented as authorization for a <code>@mutating</code> function.	Attempt to submit a <code>permit()</code> -equivalent off-chain signature that authorizes a token transfer. Consensus must reject with <code>OFF_CHAIN_SIGNATURE_FOR_MUTATING_OP</code> .
3	Build human-readable manifest display into the reference wallet	The wallet's Routing Plane must call <code>simulate()</code> on every pending transaction and render the <code>SimulationResult</code> as a human-readable summary before presenting the Approve/Reject buttons. The summary must prominently show: what leaves your wallet, what enters your wallet, what permissions you are granting, and to whom.	Test with 10 representative DeFi operations (swap, LP add, stake, borrow, etc.). Confirm the rendered summary accurately describes each operation. Test with one deliberately misleading contract and confirm the divergence flag triggers a warning.
4	Implement off-chain signature scope registry in consensus	Maintain a registry of contract functions annotated as <code>@readonly</code> . Signatures referencing these functions are valid off-chain. All other functions require on-chain transaction submission. The registry is updated through governance when new <code>@readonly</code> functions are added.	Attempt to create an off-chain signature for a function not in the registry. Reject. Attempt for a registered <code>@readonly</code> function. Accept.

2.4 Verification Tests

TEST SUITE 2A: Off-chain Signature Rejection — Create well-formed EIP-712 signatures for token transfer operations. Submit to the mempool. Every such submission must be rejected before reaching a validator, with a specific rejection reason in the RPC response.
TEST SUITE 2B: Simulation Accuracy — Build a test suite of 50 transactions with known expected asset flows. Run simulate() and compare. Measure simulation-execution divergence rate across 1,000 mainnet-equivalent transactions — target: zero divergence for honest contracts.

Attack Category 3: Transaction Signing Tricks and Ice Phishing

3.1 Threat Description

Ice phishing encompasses all attacks that manipulate users into signing malicious transactions — not by stealing their keys, but by deceiving them about what they are signing. This includes `eth_sign` (signing arbitrary hashes with no human-readable content), `SetApprovalForAll` (granting complete NFT portfolio control in one signature), and transaction simulation spoofing — where a contract behaves safely during simulation but maliciously during actual execution by checking `block.number` or `block.timestamp`. A user lost 143.45 ETH (~\$461K) to simulation spoofing in January 2025.

3.2 Synergy Implementation: Mandatory Human-Readable Manifests + Simulation Spoof Detection

Mandatory Manifests	Every contract call must include NatSpec-equivalent descriptions compiled into transaction metadata by the SynQ compiler. The protocol compares manifest descriptions against simulation results and flags mismatches before presenting the signing UI.
No Raw Hash Signing	<code>eth_sign</code> -equivalent raw hash signing is not supported at the protocol level. All signing operations produce structured, type-safe authorization objects with explicit scope declarations.
Spoof Detection	The simulation engine runs at two different block heights (current and current+1) and checks for execution divergence. Any contract that produces different results based on <code>block.number</code> or <code>block.timestamp</code> within the signing window triggers a <code>SIMULATION_DIVERGENCE</code> warning that must be acknowledged before signing.
SetApprovalForAll equivalent	SynQ's capability system requires NFT-collection approvals to enumerate the specific token IDs being delegated or to use a scoped delegation with an explicit cap. Blanket "approve all" semantics are not expressible in SynQ.

3.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Enforce NatSpec manifest requirement in SynQ compiler	The SynQ compiler must reject any function that lacks a <code>@manifest</code> annotation with at least: a plain-English description of the function's effect, a list of all state variables it modifies, and the maximum asset value it can transfer per invocation. Compilation fails without these annotations.	Attempt to compile a SynQ contract with an unannotated public function. Confirm <code>MISSING_MANIFEST</code> compilation error. Add the annotation and confirm successful compilation.
2	Implement simulation divergence detection in the VM	Extend <code>simulate()</code> to run the transaction twice: once at <code>current_block</code> , once at <code>current_block+1</code> . If the <code>SimulationResult</code> differs, set the <code>divergence_detected</code> flag. Additionally detect contracts that read <code>block.number</code> or <code>block.timestamp</code> inside execution paths that affect asset flows.	Deploy a test contract that behaves differently based on <code>block.number % 2</code> . Call <code>simulate()</code> . Confirm <code>divergence_detected = true</code> . Deploy an identical contract using a fixed constant. Confirm <code>divergence_detected = false</code> .
3	Remove raw hash signing support from protocol	Remove <code>eth_sign</code> or equivalent from the transaction type specification. The protocol must not define a transaction type that accepts an arbitrary bytes32 hash as its authorization.	Attempt to submit a transaction signed only with a raw keccak256 hash of the calldata. Confirm rejection with <code>UNSUPPORTED_SIGNATURE_TYPE</code> before mempool admission.
4	Restrict SetApprovalForAll equivalent	SynQ's collection approval syntax must require either an explicit token ID list or a capped quantity: <code>CollectionApproval{token_ids: Vec<u256>, cap: u256, expiry: u64}</code> . The compiler enforces this at the call site.	Attempt to write a SynQ contract that calls a collection-approval function with boolean 'true' as the only argument. Confirm compilation error. Rewrite with a bounded token list and confirm success.

3.4 Verification Tests

TEST SUITE 3A: Simulation Spoof Detection — Deploy 10 contracts with various block-number and timestamp dependencies in their asset-transfer logic. Confirm all 10 trigger `divergence_detected`. Deploy 10 honest contracts with deterministic behavior. Confirm zero false positives.

TEST SUITE 3B: Manifest Completeness Audit — Review all SynQ standard library functions for complete `@manifest` annotations. Establish a CI gate that fails builds when any public function lacks a valid manifest.

Attack Category 4: Signature Replay Attacks

4.1 Threat Description

Replay attacks reuse signatures valid in one context in a different context. Cross-chain replay (exploited in the \$20M Optimism OP token theft, June 2022) occurs when chain IDs are absent from signature hashes. Same-chain replay occurs when contract authorization omits nonces. ECDSA nonce reuse — where the same random k is used in two signatures — allows complete private key extraction. A December 2024 arXiv paper demonstrated that EIP-7702 delegation signatures can be replayed across every EVM chain using the same EOA, since each chain treats the authorization as fresh.

4.2 Synergy Implementation: Mandatory Scoped Signature Fields + Deterministic Nonce Generation

Mandatory fields	<code>{chain_id: u64, contract_address: bytes32, function_selector: bytes4, nonce: u64, expiry: u64}</code> — all five fields required and enforced at the consensus layer for every authorization signature
Chain-ID binding	The <code>chain_id</code> in every signature is validated against the consensus state at admission time. A signature from Synergy testnet cannot be replayed on mainnet because the <code>chain_id</code> values differ and both are checked.
Nonce management	Nonces are managed at the consensus layer, not by individual contracts. The per-account nonce is a global monotonic counter maintained in the Synergy state trie. No contract can reset, replay, or manipulate nonces.
Deterministic nonce gen	The Aegis cryptographic library enforces RFC 6979 deterministic nonce generation for all ECDSA-family signatures. This makes k -reuse mathematically impossible regardless of RNG quality, eliminating the entire class of private-key extraction via nonce reuse.
Expiry enforcement	Signatures with expiry timestamps in the past are rejected at the protocol level. Default maximum signature validity window is configurable via governance but starts at 24 hours for standard operations and 1 hour for high-value operations.

4.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Define AuthorizationSignature type in consensus	Specify the canonical binary encoding of <code>AuthorizationSignature {chain_id, contract_address, function_selector, nonce, expiry, signature_bytes}</code> . This type replaces all raw signature passing in the protocol. The encoding is fixed and versioned.	Attempt to submit a raw bytes64 signature without wrapper fields. Confirm <code>INVALID_SIGNATURE_TYPE</code> rejection. Submit <code>AuthorizationSignature</code> with each field omitted individually. Confirm <code>MISSING_SIGNATURE_FIELD</code> rejection for each.
2	Implement global nonce management in state trie	Add a per-account nonce counter to the Synergy state trie applied to all authorization operations. Implement strict monotonic advancement: the consensus layer must reject any authorization with a nonce that is not exactly (<code>current_nonce + 1</code>).	Attempt to replay a valid signed authorization with the same nonce as a previously executed operation. Confirm <code>NONCE_ALREADY_USED</code> rejection. Confirm the nonce counter is queryable via <code>rpc_getAccountNonce</code> .
3	Integrate RFC 6979 into Aegis signing library	In the Aegis cryptographic library, replace any RNG-based ECDSA nonce generation with the RFC 6979 deterministic algorithm. This must apply to all classical-family signing operations. Verify using the RFC 6979 official test vectors before integration.	Run the complete RFC 6979 test vector suite against the Aegis implementation. Zero failures required. Sign the same message 1,000 times and verify all 1,000 resulting signatures are identical (deterministic) and all verify correctly.
4	Enforce signature expiry in block validation	Validators must check signature expiry during block construction and validation. Any transaction containing an <code>AuthorizationSignature</code> with <code>expiry < block.timestamp</code> must be rejected. This check runs before execution.	Create a signature with expiry set to <code>block.timestamp + 5 seconds</code> . Wait 6 seconds. Submit the transaction. Confirm <code>SIGNATURE_EXPIRED</code> rejection.

CRITICAL NOTE: The RFC 6979 integration is not optional and has no fallback. Any signing code path that calls an RNG for nonce generation must be treated as a critical security bug. The CI pipeline must include a static analysis check that flags any import of random number generators within the Aegis signing module.

Attack Category 5: Sweeper Bots

5.1 Threat Description

Once a private key is compromised, sweeper bots monitor the mempool for any incoming transfer to the compromised address and instantly broadcast a competing transaction with higher gas to drain the funds before the owner can react. Advanced sweepers operate across multiple chains, sweep ERC-20 tokens based on live price feeds, and run indefinitely across all accounts derived from a compromised seed phrase.

5.2 Synergy Implementation: Tiered Time-Delayed Transfers + Watchdog Accounts + Panic Mode

Synergy defeats sweeper bots structurally: if funds cannot be instantly transferred, sweeper bots cannot instantly drain them. Time delays are enforced at the consensus layer and cannot be bypassed by any application or smart contract.

Tier 1 (< \$100 equiv.)	Instant execution — no delay. Standard microtransactions are not impeded.
Tier 2 (\$100 – \$10,000)	1-hour cancellation window. The transfer is committed to the ledger in a PENDING state but assets are not released to the recipient for 3,600 seconds. During this window, the sender (or an authorized watchdog) can broadcast a CANCEL_TRANSFER transaction.
Tier 3 (> \$10,000)	24-hour delay with guardian notification. An on-chain notification is automatically sent to all registered guardian accounts when a Tier 3 transfer is initiated. Any registered guardian or the watchdog account can veto.
Watchdog Account	A veto-only account that holds zero funds but has the authority to broadcast CANCEL_TRANSFER or FREEZE_ACCOUNT for its associated primary account. The watchdog cannot initiate transfers — only cancel or freeze.
Panic Mode	A PANIC_FREEZE transaction type triggered via a separate offline panic key. When broadcast, it instantly freezes all outgoing transfers from the account for a configurable duration (default: 72 hours).
Account Freeze RPC	rpc_freezeAccount and rpc_cancelPendingTransfer are first-class RPC methods serviced by all nodes within 500ms, with higher priority than standard transactions.

5.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement PENDING transfer state in consensus	Add a new transfer state PENDING to the Synergy state trie. Assets in PENDING state cannot be double-spent and do not appear in the recipient's spendable balance until the delay window elapses.	Submit a \$500 transfer. Confirm assets move to a locked pending balance. Confirm recipient cannot spend them for 3,600 seconds. Confirm they release automatically after the window.
2	Implement CANCEL_TRANSFER and FREEZE_ACCOUNT transaction types	Define these as native transaction types. CANCEL_TRANSFER references a pending transfer ID and must be signed by either the sender's key or a registered watchdog key. FREEZE_ACCOUNT must be signed by the account's panic key. Both must be executable in the same block as the transfer they are cancelling.	Register a watchdog account. Initiate a Tier 2 transfer. Sign and broadcast CANCEL_TRANSFER using the watchdog key within 30 minutes. Confirm the transfer is cancelled and assets return to the sender's confirmed balance.
3	Implement guardian notification system	When a Tier 3 transfer is initiated, the consensus layer must create notification events for all registered guardian UMA identities. These events must be queryable via <code>rpc_getPendingGuardianNotifications</code> .	Register 3 guardian accounts. Initiate a \$15,000 transfer. Confirm all 3 guardian accounts receive a notification event in the same block containing: transfer ID, amount, destination, and time remaining.
4	Implement panic key derivation in wallet	The Synergy Wallet derivation scheme must produce a dedicated panic key in a separate derivation path that cannot be inferred from the seed phrase alone. The panic key must be exportable only to a hardware security module or printed to a paper backup at wallet creation time.	Confirm the panic key is generated and displayed once for backup during setup. Confirm that possessing only the seed phrase does not reveal the panic key. Confirm PANIC_FREEZE functions correctly using the exported panic key.

IMPLEMENTATION NOTE: The 1-hour and 24-hour delay thresholds are governance-configurable defaults. Applications that require faster settlement (e.g., DEX trading) can request reduced delays by posting a bond that compensates users if a fraudulent transfer occurs.

Attack Category 6: Address Poisoning

6.1 Threat Description

Attackers send tiny transactions from addresses that match the first and last characters of the victim's regular recipients, exploiting the habit of copying addresses from transaction history. USENIX Security 2025 research documented 270 million attack attempts across Ethereum and BSC targeting 17 million unique addresses, resulting in \$83.8M in confirmed losses. The \$68M WBTC theft (May 2024) was the single largest incident.

6.2 Synergy Implementation: Protocol-Native Naming System + Anti-Spoofing Rules

Universal Account Names	Every Synergy account has a mandatory protocol-native human-readable identifier (SynID), similar to ENS but enforced by the consensus layer rather than a contract. SynID registration is required for accounts that initiate transactions above a minimum threshold. The wallet UI never displays raw hex as the primary identifier.
Address Book at Consensus	The protocol maintains per-account address books in the state trie. When a transaction is sent to a known contact, the wallet displays their SynID. When sent to an unknown address, the wallet displays a prominent UNKNOWN RECIPIENT warning.
Anti-Spoofing Rules	The consensus layer maintains a character-similarity distance metric (Levenshtein distance) for all registered SynIDs. Any attempt to register a SynID with a distance below a threshold from an existing SynID is rejected. New accounts attempting to send transactions to addresses they have never transacted with, and whose hex representation is similar to known contacts, trigger a POTENTIAL_POISONING warning.
VDF Address Cost	A Verifiable Delay Function (VDF) is required for vanity address mining — any address generation involving more than a governance-configurable number of candidates requires completing a VDF proof. This makes GPU-scale vanity address grinding economically prohibitive.
QR-Code Default	The reference wallet defaults to QR-code address transfer rather than copy/paste. A QR code is generated from the SynID (not the raw hex) and the recipient's wallet verifies the SynID matches the expected account before completing the transfer.

6.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Design and implement SynID namespace in consensus	Define the SynID format: human-readable string up to 32 characters, case-insensitive, no homoglyph characters. Store SynID registrations in the state trie. Implement SynID resolution via <code>rpc_resolveSynID</code> .	Register 'alice.syn' and attempt to register 'alice.syn' and 'alice.α' (Cyrillic α). Both variants must be rejected as homoglyph violations. Confirm <code>rpc_resolveSynID</code> returns the correct account address for 'alice.syn'.
2	Implement character-similarity anti-spoofing check	When a new SynID registration is submitted, compute the normalized Levenshtein distance against all existing SynIDs that share the same suffix. Reject registrations with distance below the governance threshold (default: 3).	Register 'exchange.syn'. Attempt to register 'exchange.syn' (lookalike Cyrillic a). Confirm rejection.
3	Implement VDF for address generation	Any wallet or node software that generates new account keys must include a VDF proof in the account registration transaction if the address was produced by iterating more than the governance parameter of keypairs. The VDF proof is verified by validators.	Attempt to register 1,000 accounts in sequence without VDF proofs. After the governance threshold, confirm registrations begin failing with <code>VDF_PROOF_REQUIRED</code> .
4	Build protocol-native address book into wallet	The wallet's Routing Plane must maintain a verified contact list backed by the consensus-layer address book. Every outgoing transaction must be checked against this list. New destinations trigger a verification flow where the user must explicitly confirm by typing (not copying) the recipient's SynID.	Send to a new address. Confirm the verification flow requires explicit SynID confirmation. Send to an address similar to a known contact. Confirm <code>POTENTIAL_POISONING</code> warning is displayed.
TEST SUITE 6: Address Poisoning Simulation — Create 100 test accounts with SynIDs one character apart from 10 commonly-used test accounts. Confirm <code>POTENTIAL_POISONING</code> warnings are displayed for all 100 and the wallet requires explicit SynID confirmation before proceeding.			

Attack Category 7: Clipboard Hijacking Malware

7.1 Threat Description

Clipboard hijackers silently replace copied cryptocurrency addresses with an attacker's address. Laplas Clipper (2022) was offered as a subscription service at \$549/year and supported 19+ cryptocurrencies. ClipBanker (2025) was distributed via fake Microsoft Office add-ins on SourceForge and monitored 2.3 million addresses. Because clipboard monitoring is invisible to the user and operates at the OS level, it bypasses all in-wallet security checks.

7.2 Synergy Implementation: Protocol Naming + Clipboard Integrity Verification

SynID as primary identifier	Since users copy-paste SynIDs (human-readable) rather than hex addresses by default, clipboard substitution of a hex address has no effect when the recipient verification flow checks against the intended SynID.
Address integrity check	When a user pastes any identifier into the wallet's recipient field, the wallet re-fetches the expected address from the intent source (address book entry, dApp-provided link, QR code) and performs a hash comparison against the pasted content. Any mismatch triggers a CLIPBOARD_MISMATCH warning.
QR-Code-first UX	The reference wallet's default UX for address transfer is QR code scanning, which completely bypasses the clipboard. The wallet generates a QR code containing the SynID (not the hex address). The receiving wallet scans and resolves the SynID. No clipboard involvement.
Intent Source Anchoring	When a dApp or external link initiates a transaction, the wallet records the intended recipient from that initiating source and compares it against what is present in the transaction object at signing time. If they differ, the wallet refuses to sign.

7.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement clipboard comparison in wallet UI	In the wallet's paste handler for any address input field: after the paste event, re-fetch the intended recipient from the authoritative source. Hash both the intended address and the pasted content. If the hashes differ, display <code>CLIPBOARD_MISMATCH</code> prominently and prevent progression.	Install a mock clipboard hijacker that substitutes addresses. Attempt to paste an address into the wallet. Confirm <code>CLIPBOARD_MISMATCH</code> appears.
2	Implement QR-code address transfer with SynID embedding	The wallet's QR code generator must embed the SynID as the primary data. The receiving wallet resolves the SynID at scan time and confirms it matches.	Modify the QR data to substitute a different hex address while keeping the same SynID. Confirm Account B's wallet rejects the scan with <code>QRCODE_RESOLUTION_MISMATCH</code> .
3	Anchor dApp-initiated transaction recipients	When a dApp submits a transaction request to the wallet, the wallet must store the intended recipient from the request in a tamper-evident session object. When the user reaches the signing step, the wallet compares the live transaction recipient against the session-stored intended recipient.	Create a dApp that initiates a transaction to Address X but modifies the transaction calldata to use Address Y before signing. Confirm the wallet detects <code>RECIPIENT_TAMPERING</code> .

Attack Category 8: Smart Contract Exploits

8.1 Threat Description

This is the broadest and most technically sophisticated category, covering reentrancy attacks (DAO hack, \$60M; Curve Finance, \$62–73M), flash loan governance attacks (Beanstalk, \$182M; Euler Finance, \$197M), MEV and sandwich attacks (\$370–500M extracted from Solana users in 16 months), bridge exploits (Ronin, \$624M; Wormhole, \$325M), and delegatecall storage collisions (Parity Wallet #2, \$154M permanently frozen). Flash loan attacks constituted 83.3% of eligible exploits in 2024.

8.2 Synergy Implementation: Multiple Structural Defenses in SynQ and at the Consensus Layer

Reentrancy — Linear Asset Types

SynQ uses a linear asset type system. Assets are values that cannot be copied or dropped — they can only be moved from one owner to another. The SynQ compiler enforces that all asset references in a function must be fully consumed or stored before any external call is made. Reentrancy is inexpressible in SynQ.

Flash Loan Governance Attacks — Hot Potato Pattern + Holding Periods

SynQ enforces the "hot potato" pattern for flash loan objects. Flash loan structs are defined without a "drop" ability — they must be consumed by a repayment function before the transaction ends. The governance module enforces a minimum token holding period: tokens acquired in the same block as a governance vote cast cannot be counted toward quorum. Single-block governance takeover is structurally impossible.

MEV and Sandwich Attacks — Encrypted Mempool + Fair Ordering

The Synergy protocol implements an encrypted mempool using threshold encryption. Transactions are encrypted with a shared public key before entering the mempool. They are decrypted only when a sufficient threshold of validators commit to the current transaction ordering. This eliminates the information asymmetry that makes frontrunning and sandwich attacks possible. A native MEV redistribution mechanism returns any detected extracted value to the transaction originators via a protocol fee adjustment.

Bridge Security — Native Cross-Chain Verification

SXCP provides native bridgeless cross-chain communication with protocol-level verification. Unlike bridge protocols that hold custodied funds in a pooled vault (the target of the Ronin and Wormhole hacks), SXCP creates no custody domain. Threshold signatures require geographically distributed validators. Mandatory time delays for large cross-chain transfers prevent instant drain scenarios even if validators are compromised.

Delegatecall — Explicit Type-Checked Interfaces

SynQ does not expose raw `delegatecall`. Module composition uses explicit, type-checked interfaces. Storage layouts are enforced at the compiler level — any module upgrade that changes the storage layout of an existing slot is a compilation error. Storage collision exploits are inexpressible in SynQ.

8.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement linear asset types in SynQ type system	Define the <code>Asset<T></code> type with linear semantics in the SynQ type checker. The type checker must enforce: (a) no Asset copy operations except through explicit controlled duplication marked <code>@splits_asset</code> ; (b) no Asset drop operations without explicit <code>@consumes_asset</code> ; (c) all external calls within a function must be preceded by a complete asset accounting proof.	Write a contract with a reentrancy pattern (send ETH, then update balance). Confirm the type checker rejects it with <code>ASSET_REFERENCE_BEFORE_EXTERNAL_CALL</code> .
2	Implement encrypted mempool with threshold decryption	Integrate a threshold encryption scheme (ECIES or a post-quantum equivalent from the Aegis suite) for mempool transaction encryption. Transactions are decrypted only when a sufficient threshold of validators commit to the current transaction ordering.	Attempt to observe another account's pending transaction content from a validator node before the decryption threshold is reached. Confirm the content is opaque. Confirm that after threshold consensus, all encrypted transactions decrypt and execute correctly.
3	Implement governance holding period in PoSy module	In the PoSy governance module, record the acquisition block for all governance tokens. When a governance vote is cast, compute the holding period for each token. Tokens with <code>holding_period < governance_minimum</code> (default: 259,200 blocks $\approx$ 15 days) are excluded from quorum calculation.	Acquire governance tokens in block N. Attempt to cast a governance vote in block N+1. Confirm the tokens are excluded from quorum. Wait until block N + <code>governance_minimum</code> . Confirm the vote is now counted.
4	Implement hot potato enforcement in SynQ for flash loans	Define <code>FlashLoan<T></code> as a linear type with no "drop" ability. The SynQ type checker must confirm that every code path that instantiates a <code>FlashLoan<T></code> also calls <code>repay(FlashLoan<T>)</code> before the transaction ends.	Write a flash loan contract that does not call <code>repay()</code> . Confirm <code>FLASHLOAN_NOT_REPAID</code> compilation error. Write one that conditionally does not repay. Confirm the type checker catches the conditional non-repayment path.
5	Implement MEV redistribution mechanism	Define a <code>ValidatorMEVReport</code> that validators submit alongside proposed blocks, listing detected MEV extraction events. The protocol fee module must compute a redistribution amount and apply it as a fee credit to the affected transaction originators in the same block.	Submit a batch of transactions including a detectable sandwich pattern. Confirm the block proposer includes a <code>ValidatorMEVReport</code> . Confirm the affected originator receives the redistribution credit in the same block.

8.4 Encrypted Mempool as Quantum On-Spend Defense

The encrypted mempool defense documented in this category serves a dual security function that must be explicitly recorded in this specification.

Primary function (as documented in v1.2): Prevention of MEV extraction, sandwich attacks, and front-running by ensuring transaction content and signing keys are not visible to validators or mempool observers until threshold decryption at block construction time.

Secondary function (quantum defense, added v1.3): Prevention of CRQC on-spend attacks. A fast-clock CRQC (superconducting or photonic architecture) could, once realized at ~500K physical qubits, break ECC-256 in approximately 9 minutes. An on-spend attack requires the attacker to observe a transaction's signing public key in the mempool, derive the private key before the transaction is finalized, and inject a forged transaction. Synergy's encrypted mempool eliminates the first requirement: signing public keys are never exposed in the Synergy mempool. This defense is independent of Synergy's ML-DSA signing (which is itself Shor-resistant) and applies specifically to the scenario where an SXCP cross-chain transaction relays a classical-key attestation through Synergy's mempool.

NOTE: *The encrypted mempool's quantum defense is additive to, not a substitute for, the CRQC on-spend protections in Category 19. Both must be maintained.*

Attack Category 9: Social Engineering and Drainer-as-a-Service

9.1 Threat Description

The industrialization of crypto phishing through Drainer-as-a-Service platforms has created a thriving criminal economy. Inferno Drainer held 40–45% market share in 2024, taking a 30% affiliate cut. North Korea's Lazarus Group, responsible for \$1.34B in crypto theft in 2024 including the Bybit hack (\$1.5B, February 2025), uses fake LinkedIn job offers as their primary vector. AI-powered deepfake scams caused \$4.6B in losses in 2024. The Ledger Connect Kit supply chain attack (December 2023) compromised 30,000+ browser downloads.

9.2 Synergy Implementation: Protocol Defenses Collectively Dismantle the DaaS Model

The DaaS business model depends entirely on the signing step being opaque and irreversible. Synergy's combination of mandatory simulation, human-readable manifests, time-delayed transfers, and scoped approvals fundamentally undermines the model: if users can clearly see what they are signing and can cancel within a window, the drainer's attack surface collapses.

Supply chain protection	SynQ compiler requires formal verification for all contracts above a configurable value threshold at deployment time. Contracts that fail formal verification cannot be deployed to mainnet, regardless of how they reached the network. This prevents backdoored smart contract libraries from being deployed.
Frontend independence	Because Synergy's security model does not depend on the correctness of frontend UIs — the protocol layer enforces simulation, manifests, and time delays regardless of what the UI shows — DNS hijacking and CDN compromise attacks cannot cause fund loss. The protocol layer will still require genuine user confirmation of the correct manifest.
Phishing site ineffectiveness	Without the permit/off-chain signature vector (blocked by Category 2 implementation) and with mandatory on-chain simulation, phishing sites cannot drain wallets through a single signature. Every drain attempt requires an on-chain transaction that the time-delay system can block.
Verification requirement	Contracts deploying above a value threshold must provide a SynQ formal verification proof as part of the deployment transaction. The consensus layer runs the proof verifier and rejects unverified deployments via the <code>--verify-deploy</code> flag.

9.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement formal verification gate for contract deployment	Define a <code>DeploymentVerificationProof</code> type required for contract deployments above the governance-set threshold. The SynQ compiler's <code>--verify-deploy</code> flag generates this proof. The consensus layer's deployment transaction handler checks for the proof and runs the on-chain verifier.	Attempt to deploy a contract above the threshold without <code>--verify-deploy</code> . Confirm <code>VERIFICATION_PROOF_REQUIRED</code> rejection. Deploy with a valid proof. Confirm acceptance.
2	Implement protocol-level audit trail for DaaS detection	The Synergy explorer must aggregate signature authorization patterns. An automated anomaly detector must flag accounts that: (a) receive approvals from more than N unique accounts within 1 hour, (b) use those approvals to transfer funds from multiple accounts within seconds of each other (sweep pattern), or (c) match known drainer contract bytecode patterns.	Deploy a test contract mimicking a drainer approval sweep. Confirm the anomaly detector flags the pattern within 2 blocks of the sweep. Confirm the on-chain alert is visible via <code>rpc_getSecurityAlerts</code> .

Attack Category 10: Multicall and Batch Transaction Abuse

10.1 Threat Description

The ERC-2771 + Multicall vulnerability (December 2023) allowed attackers to spoof any address by crafting calldata where the last 20 bytes of each sub-call contain the victim's address. The TIME token exploit used this to burn 62 billion tokens from a Uniswap pool. A separate class exploits `msg.value` reuse in delegatecall-based multicall: an attacker sends 1 ETH but executes 10 swaps each reading `msg.value = 1 ETH` — effectively spending 10 ETH worth of value with 1 ETH of input.

10.2 Synergy Implementation: Linear Payment Types + Per-Sub-call Manifests

Linear payment values	In SynQ, <code>msg.value</code> (or its equivalent) is a linear type. It is consumed on first use within a transaction. Any attempt to read <code>msg.value</code> a second time within the same transaction returns zero — the type system prevents reuse.
msg.sender integrity	SynQ's authorization model eliminates calldata-based <code>msg.sender</code> spoofing. <code>msg.sender</code> is resolved from the transaction's cryptographic authorization chain at the consensus layer, not from calldata.
Per-sub-call manifest display	Batch transactions display each sub-call's human-readable manifest separately in the wallet UI. The user sees an itemized list of every operation in the batch. Batches containing any sub-call with a missing or anomalous manifest are flagged before signing.
Batch size limit	Protocol-level maximum for batch transaction sub-call count (governance-configurable, default 20). Batches exceeding this limit are rejected at mempool admission.

10.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement linear value type in SynQ	Define <code>TransactionValue<T></code> as a linear type. The transaction executor initializes one <code>TransactionValue</code> instance per transaction. The type checker ensures this instance is consumed at most once.	Write a contract with a multicall pattern that reads <code>msg.value</code> twice. Confirm <code>VALUE_ALREADY_CONSUMED</code> type error. Rewrite with explicit value splitting. Confirm compilation success.
2	Implement UMA-based sender identity resolution	Remove all calldata-based sender inference from the SynQ execution model. The <code>tx.sender</code> binding must be populated exclusively from the <code>AuthorizationSignature</code> 's public key field, verified at the consensus layer before execution begins.	Deploy a contract that attempts ERC-2771-style sender spoofing via appended calldata bytes. Confirm the contract reads the actual authorized sender, not the appended bytes.

Attack Category 11: Upgradeable Proxy Exploits

11.1 Threat Description

Proxy patterns separate state from logic to enable upgrades, but admin key compromise enables malicious upgrades that drain every user who has ever granted approvals. Access control failures in upgradeable contracts accounted for \$953.2M in losses in 2024 — the single largest vulnerability category. The Wormhole exploit (\$325M) used a deprecated, unprotected function in an upgradeable contract. Storage layout mismatches between proxy versions (demonstrated in the Audius governance hack) can corrupt state in ways that are almost impossible to detect before exploitation.

11.2 Synergy Implementation: Protocol-Enforced Upgrade Governance + Automatic Approval Invalidation

Time-locked upgrade proposals	Contract upgrades require a time-delayed governance proposal. The default delay is 7 days. The proposal must include: the new contract bytecode hash, a diff of the storage layout changes, and a formal verification proof for the new implementation.
Multi-party upgrade approval	Upgrades require multi-party authorization using the governance quorum mechanism. No single admin key can authorize an upgrade. The threshold and quorum are set by governance and enforced by the consensus layer.
Automatic approval invalidation	When a contract upgrade is finalised, the protocol consensus layer automatically invalidates all outstanding user approvals for the upgraded contract address. Users must explicitly re-approve the new version. This is non-negotiable and cannot be disabled by the contract deployer.
Compiler-enforced storage layout compatibility	The SynQ compiler's upgrade mode verifies storage layout compatibility between the current and proposed new implementation. Any slot that changes type, shifts position, or is removed is flagged as an upgrade error. Slot additions at the end of the layout are the only permitted upgrade change.

11.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement upgrade proposal time-lock in consensus	The <code>PROPOSE_UPGRADE</code> transaction type creates an <code>UpgradeProposal</code> in the state with fields: <code>{target_contract, new_bytecode_hash, storage_layout_proof, verification_proof, quorum_threshold, unlock_time}</code> . The <code>EXECUTE_UPGRADE</code> transaction can only be processed after <code>unlock_time</code> and requires signatures from the quorum.	Submit a <code>PROPOSE_UPGRADE</code> with <code>delay=7</code> days. Attempt to submit <code>EXECUTE_UPGRADE</code> 1 day later. Confirm <code>UPGRADE_LOCKED</code> rejection.
2	Implement automatic approval invalidation on upgrade	The <code>EXECUTE_UPGRADE</code> transaction processor must iterate over all approval records for the target contract address and set their <code>expiry_timestamp</code> to <code>block.timestamp - 1</code> . This must be atomic with the upgrade itself.	Grant 100 approvals to a contract. Upgrade the contract. Confirm all 100 approvals are invalidated in the upgrade block. Confirm attempts to use any of them return <code>APPROVAL_EXPIRED</code> .
3	Implement storage layout checker in SynQ compiler	The SynQ compiler upgrade mode (<code>--upgrade-check old_abi.json new_abi.json</code>) must compare storage slot assignments and return a <code>StorageLayoutCompatibilityReport</code> . Any slot that changes type or position produces a <code>BREAKING_STORAGE_CHANGE</code> error.	Modify a contract to change a storage slot's type. Run <code>upgrade-check</code> . Confirm <code>BREAKING_STORAGE_CHANGE</code> error. Change only to add a new slot at the end. Confirm clean report.

Attack Category 12: Key Extraction and Private Key Compromise

12.1 Threat Description

Private key compromise accounts for 80.5% of the largest crypto losses in 2024. Vectors include fake wallet apps (including a Chrome extension that exfiltrated seed phrases via fake Sui blockchain transactions), mass phishing emails, hardware wallet supply chain attacks, and the Lazarus Group's fake LinkedIn job offer technique that produced the Bybit hack (\$1.5B). The damage is instantaneous and typically irreversible: an attacker with a private key can drain everything within seconds.

12.2 Synergy Implementation: Social Recovery + Key Rotation + Hierarchical Keys + Factor-Bound Root Authority

Factor-Bound Root Authority	The mnemonic-derived master seed combined with a second independent factor (hardware-backed or separately stored) is required for spending authority. An attacker who obtains only the seed phrase cannot spend. An attacker who obtains only the second factor cannot spend. Both are required.
Social Recovery (M-of-N)	The protocol-native recovery system uses an M-of-N guardian quorum. Guardians are other UMA identities. A 72-hour guardian modification delay prevents an attacker who briefly controls the account from replacing guardians before recovery can be initiated.
Key Rotation Without Address Change	Inspired by Aptos's authentication key model, Synergy allows the authentication key bound to a UMA identity to be rotated without changing the account's public address or SynID. An attacker who extracts a key can be shut out by key rotation without requiring fund migration.
Hierarchical Permission Levels	Three key tiers: (1) Daily spending key — low value operations only, policy-enforced caps; (2) High-value key — operations above policy thresholds, requires additional confirmation; (3) Recovery key — stored offline, used only to initiate recovery workflows. Compromise of the daily spending key does not grant high-value or recovery authority.
PQC Migration	The key rotation mechanism is designed to support migration from classical (ECDSA-equivalent) to post-quantum (ML-DSA / Aegis suite) without identity disruption. Keys can be rotated to PQC algorithms before quantum threats become practical.

⚠️ REQUIRED CLARIFICATION — INSERT AFTER EXISTING MIGRATION STATEMENT:
CRITICAL CLARIFICATION (v1.3): The above statement applies specifically to native Synergy keys and to users migrating from cross-chain classical keys into Synergy's identity system. It must not be read as implying that classical key usage is safe as an interim state at the protocol level. Synergy uses PQC from genesis. There is no classical key fallback at the protocol level. There is no safe interim period during which classical ECDSA keys may be used for Synergy consensus, governance, or core protocol operations. Paper 2 (Google Quantum AI, March 2026) establishes that any ECDSA key that has been used to sign a transaction exposes its public key permanently on-chain. Once a CRQC exists, every exposed classical public key is retroactively breakable. The correct model is: cross-chain classical keys are a risk surface that must be actively migrated, not a temporary bridge that can be decommissioned later at leisure.

12.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement Factor-Bound Root Authority in wallet key derivation	Modify the wallet's HD derivation scheme to require a second entropy input alongside the mnemonic. This factor is not stored in the wallet application and is not derivable from the mnemonic. Its absence makes the derived keys unusable. Implement as a KDF that takes both inputs simultaneously.	Derive spending keys using the mnemonic + factor combination. Attempt to derive the same keys using only the mnemonic. Confirm the keys are different. Confirm that possessing only the mnemonic produces keys that fail all transaction signature verification.
2	Implement M-of-N guardian recovery system	Add <code>GuardianRegistry</code> to the Synergy account state with fields: <code>guardian_list</code> (<code>Vec<UMAIdentity></code>), <code>quorum_threshold</code> (<code>u8</code>), <code>recovery_delay</code> (<code>u64</code>). Implement <code>INITIATE_RECOVERY</code> , <code>CONFIRM_RECOVERY</code> , and <code>ABORT_RECOVERY</code> transaction types.	Register 3 guardians with 2-of-3 threshold. Simulate key loss. Have 2 guardians sign <code>INITIATE_RECOVERY</code> . Confirm recovery enters <code>PENDING</code> state. Confirm it cannot execute before the delay. After delay, confirm <code>CONFIRM_RECOVERY</code> succeeds.
3	Implement key rotation without address change	The Synergy state trie must store, for each UMA identity: <code>{identity_address, current_auth_key_hash, auth_key_history[]}</code> . A <code>ROTATE_KEY</code> transaction changes <code>current_auth_key_hash</code> and appends to the history. The <code>identity_address</code> never changes.	Rotate the authentication key for a test account. Confirm the account address is unchanged. Confirm the old key no longer produces valid signatures. Confirm <code>rpc_verifyCurrentAuthKey</code> returns the new key hash.
4	Implement hierarchical key tiers in wallet	The wallet's key derivation must produce three distinct keys: spending key (path <code>m/44'/chain'/0'/0/0</code>), high-value key (<code>m/44'/chain'/0'/0/1</code>), recovery key (<code>m/44'/chain'/0'/0/2</code>). The Local Policy Engine enforces which key tier can authorize which operation type.	Attempt a high-value transaction with only the spending key. Confirm <code>INSUFFICIENT_KEY_TIER</code> rejection. Approve with the high-value key. Confirm success.

CRITICAL SECURITY REQUIREMENT: The 72-hour guardian modification delay is not configurable by individual users without governance approval. An attacker who gains brief access cannot accelerate this delay. The delay implementation must be in the consensus layer, not in wallet software.

Attack Category 13: Emerging and Novel Attack Vectors

13.1 Threat Description

Several emerging attack classes exploit newer protocol features or governance mechanisms. CREATE2 opcode abuse allows pre-computation of contract addresses before deployment, enabling attackers to obtain approvals for non-existent addresses that security tools cannot flag (\$60M stolen, 99,000 victims). ERC-4337 + EIP-7702 interactions create persistent execution takeover opportunities. Token callback attacks (ERC-777, ERC-1155 transfer hooks) enable reentrancy via mechanisms that bypass standard guards. Abandoned DAO governance attacks (Beanstalk, \$182M via flash-loaned governance tokens) exploit insufficient governance participation protections.

13.2 Synergy Countermeasures

CREATE2 equivalent	Synergy requires all contract addresses to be registered in the SynID namespace before they can receive approvals. The protocol rejects approvals targeting non-existent (unregistered) addresses. This eliminates pre-deployment approval collection.
Token callback attacks	SynQ does not support arbitrary callbacks during token transfers. Asset transfers are atomic consensus-layer operations. No user-defined code executes during an asset transfer. Callbacks that need to respond to transfers must use a pull-based pattern where the recipient explicitly queries for incoming transfers.
Governance protection	Protocol-enforced minimum governance token holding periods prevent flash-loan governance attacks. Additionally, governance proposals require multi-block voting periods with quorum requirements that cannot be satisfied in a single block regardless of token holdings.
RPC integrity	The Synergy light client protocol includes cryptographic proofs (state roots) for all state queries. Any state query response that does not include a valid proof against the latest finalised block header is considered potentially malicious. The reference wallet verifies all RPC state proofs before using query results.
ERC-4337 equivalent	Synergy's account abstraction equivalent uses SCETP ingress with explicit authority chains rather than bundler-mediated UserOperations. Every execution request carries a cryptographic provenance chain traceable to a UMA identity. Bundler-type relayers cannot forge authority.

13.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement pre-deployment registration requirement	Add a REGISTER_CONTRACT_ADDRESS transaction type that must precede any approval targeting that address. The consensus layer rejects ApprovalStruct transactions where the spender field references an address not in the contract registry.	Attempt to create an approval targeting an address with no transaction history. Confirm APPROVAL_TARGET_NOT_REGISTERED rejection. Register the address first. Confirm the approval is then accepted.
2	Enforce no-callbacks during asset transfers	In the SynQ VM execution model, asset transfer operations are classified as CONSENSUS_ATOMIC, meaning they complete fully before any user contract code resumes execution.	Deploy a contract that attempts to re-enter via a transfer callback. Confirm the re-entry attempt finds the state already updated and that any callback hook is called only after the transfer completes.
3	Implement RPC state proof verification in light client	The Synergy light client must request state proofs (Merkle proofs against the latest block's state root) for all account and storage queries. The client verifies these proofs locally before using any RPC response. Implement rpc_getStateWithProof as the canonical state query method.	Run a test node that returns falsified account balances. Connect the reference wallet as a light client. Attempt a transaction. Confirm the wallet detects the proof verification failure and refuses to proceed.
4	Implement multi-block governance voting window	In the PoSy governance module, proposals must remain open for a minimum voting period measured in blocks (default: 21,600 blocks ≈ 30 days). Block-level vote weight snapshots are taken at proposal creation time, preventing flash-loan-acquired tokens from influencing votes.	Acquire a large amount of governance tokens after the snapshot. Attempt to vote with those tokens. Confirm they are excluded from quorum calculation.

Attack Category 14: EIP-7702 Delegation Exploitation

14.1 Threat Description and Why This Is Categorically Different

EIP-7702 was activated on Ethereum mainnet as part of the Pectra upgrade in May 2025. It introduces a new transaction type (type 0x04) that allows an EOA to temporarily set its bytecode to point at a smart contract's code — effectively letting any wallet become a smart account without redeployment. The attack surface this creates is categorically more dangerous than every other phishing vector in this document combined. Every other category exploits a permission to do one specific thing. EIP-7702 delegation exploitation gives an attacker the ability to replace the victim's account with attacker-controlled contract code. That is not "steal my tokens." That is "become me, permanently, across every chain I exist on, at zero ongoing cost."

Five Distinct Attack Vectors

Vector 1 — Delegation Signature Phishing	An EIP-7702 authorization tuple has the form (chain_id, delegate_address, nonce). To an unsuspecting user, a request to sign this tuple looks identical to the permit-style "sign this message" requests they have been conditioned to accept for gasless transactions. When the attacker broadcasts the signed tuple as part of a type-0x04 transaction, the victim's EOA is delegated to execute the attacker's contract code for all subsequent operations. Unlike a permit signature that authorizes a single transfer, a successful delegation turns the victim's EOA into an attacker-controlled execution environment for every future transaction.
Vector 2 — Cross-Chain Metastasis via chainId=0	A December 2024 arXiv paper demonstrated that EIP-7702 authorization tuples signed with chain_id=0 are treated as valid by every EVM chain where the signing address exists. One phishing signature simultaneously compromises the victim's account on Ethereum mainnet, all EVM-compatible L2s (Arbitrum, Optimism, Base, Polygon), any sidechains, all testnets, and any future EVM chain the address is later used on. The practical consequence: one successful phishing interaction can produce losses across every chain simultaneously.
Vector 3 — Persistent Execution Takeover	In some EIP-7702 implementation interpretations, the code delegation persists beyond the initiating transaction depending on whether the delegated contract resets or preserves the code slot. This creates an ongoing execution takeover where the attacker's code runs every time the compromised EOA is the execution context — including when called by other smart contracts that have legitimate approved interactions with the EOA.
Vector 4 — Paymaster-Funded Unlimited Exploitation	When combined with ERC-4337's account abstraction model, EIP-7702 creates a dangerous amplification: paymasters can sponsor gas costs for bundled UserOperations targeting the compromised account. Because the attacker pays no gas, they can submit an unlimited number of exploitation attempts at zero cost. This turns account compromise from a one-time event into an ongoing, economically unbounded attack loop.
Vector 5 — Social Engineering via "Account Upgrade" Framing	EIP-7702's legitimate use case — enabling "smart account" features for standard wallets — creates a powerful social engineering framing. Phishing sites present delegation requests as "upgrade your wallet to access advanced features," "enable gasless transactions," or "connect to our exclusive DeFi protocol." Users who understand EIP-7702 are actually more susceptible to this framing, not less, because the underlying technology is real and desirable.

14.2 Why Synergy Is Structurally Immune

No EOA Code Slot Exists	Synergy's account model does not have an EIP-7702 equivalent mechanism. There is no "code slot" attached to an identity that can be overwritten by a signed tuple. The three-plane wallet model separates Identity, Routing, and Execution by design. The Identity Plane never executes transactions and has no code slot to delegate. Delegation-style attacks have no surface to target at the account model level.
chainId=0 Explicitly Rejected	Synergy's <code>AuthorizationSignature</code> type requires <code>chain_id</code> to be a non-zero value matching the current chain. The consensus layer rejects any authorization where <code>chain_id</code> is zero. The cross-chain metastasis attack vector does not exist in Synergy.
UMA Identity Decoupling	In Synergy, identity is not equivalent to key material. Even if an attacker could obtain a delegation-equivalent signed authorization, it would not grant them the victim's UMA identity, which is the actual authorization substrate governing recovery rights, governance weight, contract authority, and policy state. Delegating a key does not delegate the identity, so the attacker gains execution capability for specific operations but not full account control.
Local Policy Engine as Hard Gate	Any operation that modifies execution authority must pass through the Local Policy Engine before any signing occurs. The Policy Engine's default rules classify execution authority modification as a high-value operation requiring the high-value key tier, explicit guardian notification, and a mandatory 24-hour delay. An attacker cannot trick a user into signing a delegation through a phishing site because the policy gate blocks it before the signing UI is ever reached.
Mandatory Delegation Manifest	The SynQ compiler and wallet policy engine treat "execution delegation" as a class of operation that requires a human-readable manifest explicitly stating: the delegate contract address, the scope of delegated authority, the duration, and the exact operations the delegate can perform on the user's behalf. The manifest must be confirmed by the user reading it — not just clicking Accept.
SCETP Replaces Bundler Model	Synergy's account abstraction uses SCETP ingress with explicit cryptographic authority chains rather than ERC-4337 bundler-mediated UserOperations. There is no paymaster concept in Synergy's native execution model that can sponsor unlimited free attack attempts. Every execution request traces to a UMA identity via a verifiable chain. The paymaster-funded unlimited exploitation vector (Vector 4) has no entry point.

14.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Explicitly prohibit chainId=0 in all authorization signatures	In the <code>AuthorizationSignature</code> validation logic, add a dedicated pre-check that rejects any signature where <code>chain_id == 0</code> with error code <code>INVALID_CHAIN_ID_ZERO</code> . This check must run before any other signature validation, before mempool admission.	Submit a well-formed <code>AuthorizationSignature</code> with <code>chain_id=0</code> . Confirm <code>INVALID_CHAIN_ID_ZERO</code> rejection at the mempool admission stage.
2	Define "execution authority modification" as a protected operation class	In the Local Policy Engine specification, define a new operation class: <code>EXECUTION_AUTHORITY_MODIFICATION</code> . Any operation that modifies who can execute code on behalf of a Synergy account falls into this class. Assign this class the highest protection tier: requires high-value key, requires guardian notification, requires 72-hour time-lock, requires explicit manifest review.	Attempt to submit a delegation-type operation via a dApp using only the spending key. Confirm policy rejection with <code>INSUFFICIENT_KEY_TIER_FOR_AUTHORITY_MODIFICATION</code> .
3	Build a Delegation Registry in the consensus state trie	Add a <code>DelegationRegistry</code> to the Synergy state that tracks all active execution delegations per account: <code>{delegator_UMA, delegate_address, scope_hash, expiry, creation_block, guardian_quorum_proof}</code> . Implement <code>rpc_getActiveDelegations</code> as a first-class RPC method.	Establish a test delegation. Query <code>rpc_getActiveDelegations</code> . Confirm the delegation appears with all fields. Confirm <code>DELEGATION_CREATED</code> event was emitted.
4	Enforce automatic delegation revocation on key rotation	In the key rotation implementation (Category 12, Step 3), extend the <code>ROTATE_KEY</code> transaction processor to also invalidate all active delegation records for the rotating account. Delegation re-establishment after key rotation requires fresh guardian authorization.	Establish 3 active delegations on an account. Rotate the authentication key. Confirm all 3 delegations appear as <code>REVOKED</code> in the registry with the rotation block number.
5	Implement delegation scope verification in the SynQ compiler	When a SynQ contract declares that it can operate as a delegate (via an <code>@accepts_delegation</code> annotation), the compiler must verify that the contract's authority scope is bounded. Contracts that attempt to accept delegation with unbounded scope fail compilation with <code>UNBOUNDED_DELEGATION_SCOPE</code> .	Write a delegating contract with no scope constraints. Confirm <code>UNBOUNDED_DELEGATION_SCOPE</code> compilation failure. Add explicit scope constraints. Confirm compilation success.
6	Build cross-chain delegation alert system for the EVM compatibility lane	For the EVM-compatible execution surface, implement a transaction pre-screening layer that detects type-0x04 transaction patterns (EIP-7702-equivalent structures from external chains) and routes them through the same <code>EXECUTION_AUTHORITY_MODIFICATION</code> policy class.	Submit a well-formed Ethereum type-0x04 transaction to the Synergy EVM RPC endpoint. Confirm it is caught by the pre-screening layer and routed through <code>EXECUTION_AUTHORITY_MODIFICATION</code> policy.

14.4 Verification Tests

TEST SUITE 14A: Cross-Chain Replay Prevention — Generate valid EIP-7702 authorization tuples with chain_id=0 as a test vector. Attempt submission to Synergy nodes directly, via the EVM compatibility endpoint, and embedded inside batch transactions. All three paths must reject with INVALID_CHAIN_ID_ZERO. Rejection must occur before mempool admission, not at execution time.
TEST SUITE 14B: Policy Engine Gate Coverage — Attempt delegation operations via: (1) a malicious dApp that does not present a manifest; (2) a manifest that understates the scope; (3) a delegation signed by only the spending key tier; (4) a delegation without guardian notification; (5) a delegation with a fake guardian quorum proof. All five must be blocked before any signing UI is presented to the user.
TEST SUITE 14C: Delegation Registry Integrity — Establish 10 delegations with varied scopes and expiry times. Trigger key rotation for 5 of the 10 accounts. Confirm all 5 delegations are revoked in the rotation block. For the remaining 5, allow delegations to expire naturally and confirm DELEGATION_EXPIRED events are emitted at the correct blocks.
TEST SUITE 14D: Paymaster Attack Simulation — Attempt to replicate the paymaster-funded unlimited exploitation pattern against a Synergy account using the SCETP ingress model. Confirm that without a valid cryptographic authority chain traceable to the account's UMA identity, the paymaster-equivalent cannot sponsor execution.
EIP-7702 CRITICAL DESIGN NOTE: Because EIP-7702 is live on Ethereum mainnet, the Synergy EVM compatibility lane requires special attention. Any developer migrating an Ethereum application to Synergy that uses type-0x04 transactions for legitimate account abstraction purposes must use the Synergy-native SCETP mechanism instead.

Attack Category 15: Token Impersonation, Scam Assets, and Duplicate Asset Exploits

15.1 Threat Description

Unlike exploits that target execution logic, token impersonation attacks exploit **user perception and asset identity ambiguity**.

Attackers deploy tokens or collections that:

- Copy names, symbols, or branding of legitimate assets
- Mimic verified token metadata or NFT collections
- Present themselves as “official,” “wrapped,” or “native” assets
- Exploit wallet UI limitations to appear indistinguishable from legitimate tokens

These attacks do not require vulnerabilities in smart contracts or private key compromise. They rely on **identity confusion**, which is fundamentally a protocol design failure if left unaddressed.

On most blockchain networks, tokens are identified primarily by:

1. name
2. symbol

Both are trivial to duplicate.

As a result, users are routinely exposed to:

1. fake stablecoins
2. fake wrapped assets
3. impersonation NFTs
4. malicious airdrop tokens
5. phishing-linked token contracts

This category represents a **systemic failure in asset identity modeling**, not a user error.

15.2 Synergy Implementation: Canonical Asset Identity + Protocol-Enforced Authenticity

Synergy eliminates token impersonation at the architectural level by enforcing **canonical identity, registry verification, and wallet-level enforcement rules**.

Core Principle: Identity Is Not Symbol-Based

Token identity SHALL NOT be derived from:

1. name
2. symbol
3. metadata
4. logo

Instead, identity is defined as:

CanonicalIdentity

contract_address

issuer_address

genesis_metadata_hash

chain_id

)

=

hash(

+

+

+

This identity is immutable and globally unique.

Protocol-Level Asset Registry

Synergy SHALL maintain a **canonical asset registry** with the following properties:

- 1. Each verified asset is bound to its canonical identity
- 2. Registry entries include:
 - 1. issuer attestation
 - 2. metadata hash
 - 3. verification status
 - 4. asset classification
- 3. Registry is:
 - 1. cryptographically verifiable
 - 2. queryable via RPC
 - 3. consensus-anchored or Merkle-root committed

Unregistered assets are considered **unverified by default**.

Issuer-Bound Genesis Attestation

Every token deployment **MUST** include a signed genesis attestation:

```
{
  contract_address
  issuer_address
  token_name
  token_symbol
  decimals
  metadata_hash
  chain_id
  creation_tx_hash
}
```

This attestation:

- 1. is signed by the issuer
- 2. is permanently recorded
- 3. is used for verification and registry inclusion

Symbol and Name Protection

The protocol enforces:

- 1. Reserved symbol protection (e.g., SNRG)
- 2. Confusable name detection:
 - 1. homoglyph filtering
 - 2. Levenshtein distance threshold
 - 3. normalization rules

Registrations violating these rules are rejected.

Token Fingerprint Requirement

Every asset MUST expose a deterministic fingerprint:

```
asset_fingerprint                                     = keccak256(  
  contract_address                                   ||  
  issuer_address                                     ||  
  metadata_hash                                       ||  
  chain_id                                           )
```

Wallets, explorers, and APIs MUST use this fingerprint as the primary identity key.

Verified vs Unverified Asset Enforcement

Wallet behavior is strictly defined:

Verified Assets:

- 1. MAY display symbol prominently
- 2. MAY hide full address by default
- 3. MAY be auto-discovered

Unverified Assets:

- 1. MUST display full contract address
- 2. MUST display issuer address
- 3. MUST show UNVERIFIED warning
- 4. MUST NOT auto-display balances without user opt-in
- 5. MUST require manual import

Discoverability Restrictions

The protocol enforces:

- 1. Default indexing = **verified assets only**
- 2. Unverified tokens:
 - 1. excluded from default wallet views
 - 2. excluded from automatic balance scanning
 - 3. excluded from token search results unless explicitly requested

This eliminates **spam token airdrop visibility attacks**.

Pre-Deployment Registration Requirement (Optional Enforcement Mode)

For high-security deployments, the protocol MAY require:

- 1. contract address pre-registration
- 2. issuer identity binding before deployment
- 3. registry reservation prior to token creation

This prevents:

- 1. CREATE2-style impersonation

- 2. pre-approval scams

Revocation and Dispute Lifecycle

The registry MUST support:

- 1. asset reporting
- 2. dispute review
- 3. verification downgrade
- 4. revocation

States:

VERIFIED → FLAGGED → UNDER_REVIEW → REVOKED

Wallets MUST reflect state transitions immediately.

Cross-Chain Asset Authenticity

For external assets (e.g., USDC):

- 1. canonical issuer mapping MUST be defined
- 2. wrapped or mirrored assets MUST:
 - 1. declare origin chain
 - 2. declare issuer proof
 - 3. bind to cross-chain identity mapping

Unverified “wrapped” assets MUST be flagged as:

UNVERIFIED_EXTERNAL_ASSET

15.3 Implementation Steps

Step	Action	Implementation Detail	Verification Test
1	Implement canonical identity hashing	Add asset_fingerprint to token metadata and RPC responses	Deploy 2 tokens with identical names; confirm distinct fingerprints
2	Build registry service	On-chain or anchored registry storing verified assets	Query registry; confirm correct verification states
3	Enforce wallet display rules	Modify wallet UI to enforce verified/unverified behavior	Import scam token; confirm warning + restricted display
4	Implement confusable detection	Add normalized string comparison in registration logic	Attempt homoglyph token; confirm rejection
5	Add discovery filtering	Index only verified tokens by default	Airdrop spam token; confirm not shown in wallet
6	Implement revocation system	Add registry state transitions and events	Flag token; confirm downgrade in wallet

15.4 Verification Tests

TEST SUITE 15A: Duplicate Token Detection	
Deploy multiple tokens with identical: 1. names 2. symbols 3. metadata	Confirm: 1. only one can be verified 2. others are flagged as unverified
TEST SUITE 15B: Wallet Enforcement	
Inject: 1. scam token 2. fake stablecoin 3. impersonation NFT	Confirm: 1. all display UNVERIFIED 2. full addresses shown 3. no auto-balance display
TEST SUITE 15C: Confusable Name Rejection	
Attempt registrations using: 1. homoglyphs 2. spacing tricks 3. case variations	Confirm: 1. all rejected
TEST SUITE 15D: Registry Revocation	
Mark a verified token as compromised. Confirm: 1. status changes to REVOKED 2. wallets update immediately 3. transfers trigger warnings	

15.5 Security Outcome

This system eliminates:

- 1. fake tokens
- 2. impersonation assets
- 3. scam airdrops
- 4. fake “official” assets
- 5. UI-based asset spoofing

by making them:

- 1. non-discoverable
- 2. non-trusted
- 3. non-confusable
- 4. visibly flagged

Master Reference: Protocol-Level Security Features

The following table provides a complete reference of all protocol-level security features described in this document, mapped to the attack categories they address.

Protocol Feature	Implementation Mechanism	Attack Categories Defeated	Consensus?	SynQ?
Mandatory Scoped Approvals	5-field struct enforced at consensus; expiry required; revokeAll() native tx type	Cat. 1: ERC-20 Approvals	YES	YES
Protocol-Native Intent System	On-chain submission required; simulate() opcode; off-chain signatures read-only	Cat. 2: Permit Phishing	YES	YES
Mandatory Transaction Manifests	NatSpec enforced by compiler; comparison against simulation results	Cat. 3: Ice Phishing	NO	YES
Simulation Divergence Detection	Dual-height simulation; block.number/timestamp dependency flags	Cat. 3: Ice Phishing	YES	YES
Scoped Signature Fields	{chain_id, contract, selector, nonce, expiry} mandatory in all signatures	Cat. 4: Replay Attacks	YES	YES
RFC 6979 Deterministic Nonces	Enforced in Aegis cryptographic library; RNG banned from signing paths	Cat. 4: ECDSA k-reuse	NO	NO (crypto lib)
Time-Delayed Transfers (3-tier)	PENDING state in consensus; configurable thresholds; guardian notification	Cat. 5: Sweeper Bots	YES	NO
Watchdog Accounts	Veto-only role; CANCEL_TRANSFER authority; no fund-holding requirement	Cat. 5: Sweeper Bots	YES	NO
Panic Mode / PANIC_FREEZE	Separate offline panic key; instant account freeze tx type	Cat. 5, 12: Active Attacks	YES	NO
SynID Naming System	Protocol-enforced human-readable IDs; homoglyph rejection; VDF address cost	Cat. 6: Address Poisoning	YES	NO
Anti-Spoofing Rules	Character similarity threshold enforcement; NEW_SIMILAR_ADDRESS warnings	Cat. 6: Address Poisoning	YES	NO
Address Book at Consensus	Per-account contacts in state trie; UNKNOWN_RECIPIENT warnings	Cat. 6, 7: Poisoning & Hijacking	YES	NO
Clipboard Integrity Verification	Re-fetch from intent source; hash comparison; QR-code-first UX	Cat. 7: Clipboard Hijacking	NO	NO (wallet)

Protocol Feature	Implementation Mechanism	Attack Categories Defeated	Consensus?	SynQ?
Linear Asset Types	Assets cannot be copied/dropped; compiler enforces pre-call accounting	Cat. 8: Reentrancy	NO	YES
Encrypted Mempool	Threshold encryption; decrypt only at consensus ordering time	Cat. 8: MEV/Sandwich/Frontrun	YES	NO
Governance Holding Periods	Token acquisition block recorded; flash-loan votes excluded from quorum	Cat. 8, 13: Flash Loan Gov.	YES	NO
Hot Potato Flash Loan Pattern	FlashLoan<T> has no drop ability; repayment consumption enforced by types	Cat. 8: Flash Loan Attacks	NO	YES
No Raw Delegatecall	Module composition via explicit type-checked interfaces only	Cat. 8: Delegatecall Exploits	NO	YES
Native Cross-Chain Verification	SXCP bridgeless; threshold signatures; time delays for large transfers	Cat. 8: Bridge Exploits	YES	YES
Formal Verification Gate	DeploymentVerificationProof required above value threshold	Cat. 9: DaaS / Supply Chain	YES	YES
Linear Payment Values	msg.value linear type; consumed on first use; reuse is type error	Cat. 10: Multicall Abuse	NO	YES
UMA-Based Sender Identity	tx.sender from AuthorizationSignature; calldata manipulation has no effect	Cat. 10: Sender Spoofing	YES	YES
Upgrade Governance + Time-Lock	7-day default delay; multi-party quorum; verification proof required	Cat. 11: Proxy Exploits	YES	NO
Upgrade-Triggered Approval Invalidation	Automatic invalidation of all approvals on contract upgrade, atomic in same block	Cat. 11: Malicious Upgrades	YES	NO
Storage Layout Compatibility Check	Compiler upgrade-mode detects breaking storage changes	Cat. 11: Storage Collisions	NO	YES
Factor-Bound Root Authority	Seed phrase + independent factor required; neither alone grants spend authority	Cat. 12: Key Extraction	NO	NO (wallet)
Social Recovery M-of-N Guardians	Guardian quorum; 72h modification delay; explicit recovery state machine	Cat. 12: Key Compromise	YES	NO

Protocol Feature	Implementation Mechanism	Attack Categories Defeated	Consensus?	SynQ?
Key Rotation Without Address Change	Current auth key hash mutable; identity address stable	Cat. 12: Key Compromise	YES	NO
Hierarchical Key Tiers	Spending / high-value / recovery keys at separate derivation paths	Cat. 12: Key Compromise	NO	NO (wallet)
Pre-Deployment Registration	Contract address registry; approvals to unregistered addresses rejected	Cat. 13: CREATE2 Abuse	YES	NO
No Transfer Callbacks	Asset transfers are CONSENSUS_ATOMIC; no user code during transfer	Cat. 13: Callback Reentrancy	YES	YES
RPC State Proof Verification	Merkle proofs for all state queries; verified by light client before use	Cat. 13: Compromised RPC	YES	NO
Multi-Block Governance Window	Minimum 30-day voting period; vote weight snapshotted at proposal creation	Cat. 13: Abandoned DAO Attacks	YES	NO
chainId=0 Prohibition	Explicit consensus rejection of any AuthorizationSignature where chain_id==0	Cat. 14: EIP-7702 Cross-Chain Replay	YES	NO
Execution Authority Modification Class	Policy Engine classifies delegation ops as highest protection tier; high-value key + guardian + 72h delay	Cat. 14: EIP-7702 Delegation Phishing	YES	NO (wallet)
Delegation Registry	Active delegations tracked in state trie; DELEGATION_CREATED/REVOKED events	Cat. 14: EIP-7702 Takeover	YES	NO
Auto-Revocation on Key Rotation	ROTATE_KEY processor atomically invalidates all active delegations	Cat. 14: EIP-7702 Persistence	YES	NO
Bounded Delegation Scope	@accepts_delegation requires declared selector list, value cap, and expiry	Cat. 14: EIP-7702 Scope Abuse	NO	YES
Mandatory Delegation Manifest	Human-readable display of delegate address, authority scope, duration	Cat. 14: EIP-7702 Social Engineering	NO	NO (wallet)
EVM Compatibility Lane Screening	type-0x04 pattern detection routes to EXECUTION_AUTHORITY_MODIFICATION class	Cat. 14: EIP-7702 via EVM lane	YES	NO

Master Reference: SynQ Language Security Guarantees

The following table details every vulnerability class that SynQ eliminates by design at the compiler and type-system level. These are not runtime checks — they are guarantees that apply to every contract written in SynQ, regardless of developer skill or security awareness. A contract that could be vulnerable cannot compile.

SynQ Feature	Vulnerability Class Eliminated	Implementation Mechanism	Attack Categories
Linear asset types (copy/drop disabled)	Double-spending, resource leaks, reentrancy, unaccounted tokens	Type checker rejects any code that copies or drops an Asset<T>; all asset paths must terminate in consume or store	Cat. 1, 8
Mandatory approval expiry	Stale infinite approvals, approval farming, dormant attack surface	Compiler rejects ApprovalStruct without expiry field; no default expiry is assigned	Cat. 1, 2
Scoped approvals (function + params + amount + time)	Over-permissioning, approval-based draining beyond scope	ApprovalStruct fields are enforced by compiler; protocol verifies scope on every use	Cat. 1, 2
Built-in overflow protection	Integer overflow/underflow exploits (no SafeMath library needed)	All arithmetic operations auto-checked at compile time; no unchecked arithmetic keyword	Cat. 8
Language-level reentrancy prevention	All reentrancy variants including cross-contract reentrancy	Mutex on all external calls enforced by compiler; assets must be stored before external call	Cat. 8
No raw delegatecall	Storage collision, delegatecall exploits, shared library attacks	SynQ has no delegatecall opcode; module composition uses explicit typed interfaces	Cat. 8, 11
Bounded loops only	Gas limit attacks, denial of service via unbounded iteration	Compiler rejects loops without a provable upper bound; all recursion must be bounded	Cat. 8, 13
No inheritance (composition only)	Hidden vulnerabilities from complex inheritance hierarchy, function shadowing	SynQ modules compose via explicit interface references; no implicit method resolution	Cat. 8, 11
Module-scoped resource access	Unauthorized resource access, privilege escalation via external calls	Resources are only accessible within the module that declares them	Cat. 8, 9

SynQ Feature	Vulnerability Class Eliminated	Implementation Mechanism	Attack Categories
Asset conservation laws	Lost tokens, accounting errors, mint/burn without authorization	Compiler and runtime verify total asset supply is conserved across all execution paths	Cat. 1, 8
Built-in formal verifier	Logic errors, specification violations, invariant breaks	Move Prover-style verification integrated into SynQ compiler; failures block compilation	Cat. 8, 9, 11
Human-readable manifests (required)	Blind signing, ice phishing, opaque transaction approval	Compiler rejects public functions without @manifest annotation	Cat. 3
Badge-based authorization	Address spoofing, caller impersonation via calldata manipulation	Authorization checked against capability badges not addresses	Cat. 6, 10
Hot potato pattern (flash loans)	Flash loan default, single-block governance flash attacks	FlashLoan<T> struct has no drop ability; type checker ensures repayment	Cat. 8
No arbitrary callbacks during transfers	ERC-777/ERC-1155 callback reentrancy, token hook attacks	Transfers are consensus-atomic; no user code executes during asset movement	Cat. 13
Deterministic execution	Unexpected failures, state divergence, simulation spoofing	Same transaction always produces same result; block.number/timestamp cannot affect asset flows in certified contracts	Cat. 3
Explicit authority declaration	Implicit privilege escalation, call-context authority leaks	All state transitions declare who is acting, what role, and under what conditions	Cat. 8, 9, 10
Explicit failure modelling	Silent failures, partial side effects, ambiguous rollback	All failure cases handled by type system; no code path produces partial side effects	Cat. 8, 13

Overall Verification Framework

Security implementation without systematic verification is theatre. The following framework defines how the engineering team will verify that every security measure in this document actually works under adversarial conditions — not just under ideal conditions. The framework has four levels.

Level 1 — Unit Tests (Per-Feature)

Every feature described in this document must have associated unit tests that confirm the basic enforcement behavior. These are specified within each attack category section above. Unit tests must be automated and run as CI gate checks on every commit to the protocol repository. No code may merge to main that reduces the unit test pass rate below 100% for security-critical test suites.

Level 2 — Adversarial Integration Tests (Per Attack Category)

Each of the 18 attack categories must have an adversarial test suite that attempts the actual attack flow — not just the mitigation. For example, the reentrancy test suite must deploy a contract that genuinely attempts reentrancy, confirm it cannot compile in SynQ, and (for the EVM compatibility lane) confirm that the protocol-level defenses prevent execution even if the attack is expressed via the EVM compatibility interface.

Level 3 — Formal Verification (Protocol Invariants)

The following properties must be formally proven before mainnet launch: (1) linear asset conservation — no SynQ execution path can create or destroy assets without explicit authorization; (2) approval scope enforcement — no approval can authorize operations outside its declared scope; (3) time-delay bypass impossibility — no sequence of consensus-valid transactions can cause a pending transfer to release before its delay window; (4) guardian modification delay — no governance action can reduce the guardian modification delay for any account without that account's own explicit authorization.

Level 4 — External Security Audit

Before testnet launch, a third-party security audit must be commissioned for: the SynQ compiler and type checker (with specific focus on linear type enforcement and authority declaration soundness), the consensus-layer enforcement modules (approval validation, time-delay state machine, encrypted mempool decryption), and the Synergy Wallet's three-plane model and Local Policy Engine. The audit scope must explicitly include adversarial tests for each of the 18 attack categories in this document. Audit findings are classified as Critical, High, Medium, or Low. No Critical or High findings may remain unresolved at testnet launch.

SECURITY COMMITMENT: The total addressable attack surface documented across all 14 baseline categories exceeds \$10 billion in confirmed losses. A protocol that eliminates even 70% of these vectors — not through education, not through tooling, but through architectural design — represents a generational improvement in blockchain security. The measures in this document are designed to eliminate the structural conditions that make these attacks possible, not merely to make them harder.

Security Implementation Specification

Directed Acyclic Graph (DAG) Extension

Version	1.1 — DAG extended normative companion to the baseline Synergy security specification.
Classification	Internal technical specification for architecture, protocol engineering, validator operations, audit, and red-team execution.
Scope	Security rules and enforcement requirements for Synergy's cluster-local certified DAG sublayer, including dissemination, availability, ordering, replay, pruning, recovery, RPC exposure, and forensic retention.
Authority	This document extends the baseline Security Implementation Specification v1.0 and is intended to be applied together with the DAG Architecture & Security Specification v1.0 and the DAG RPC Specification (Part 3).
Primary Settlement Rule	The DAG is not an alternate finality mechanism. PoSy dual-quorum quorum certificates remain the only canonical settlement artifact.

SECURITY CRITICAL — INTERNAL USE ONLY — CONFIDENTIAL

FAIL-CLOSED DAG RULE: If DAG correctness, availability certification, checkpoint lineage, or deterministic ordering cannot be proved locally, the validator must refuse DAG-influenced ordering and enter quarantine or safe fallback rather than continue optimistically.

DAG Extension: Executive Summary

Synergy's cluster-local DAG is the right architectural move only if it remains tightly bounded. The wrong design would be to let the DAG become a second settlement plane, to treat raw vertex gossip as authoritative, or to accept availability claims that are not backed by durable batch possession. This specification therefore codifies one blunt principle: the DAG may improve dissemination, batching, and proposer input quality, but it must never dilute finality, auditability, or fail-closed behavior.

DAG failures are different from ordinary contract-layer failures. The dangerous faults are mostly network-plane and coordination-plane faults: withholding, false receipts, missing-parent poisoning, clique formation, replay across epochs, deterministic-order divergence, pruning corruption, and control-plane abuse. Those faults do not show up as a neat contract revert; they manifest as disagreement about what data exists, what can be ordered, and what evidence survives after the fact.

This update adds three DAG-specific attack categories to the Synergy security model. Attack Category 16 covers dissemination and availability attacks. Attack Category 17 covers ordering, replay, equivocation, and cross-layer consistency attacks. Attack Category 18 covers pruning, recovery, forensics, and RPC/control-plane abuse.

Relationship to the Baseline Security Specification

The baseline Security Implementation Specification remains in force. Every pre-existing consensus, wallet, language, approval, replay, recovery, and operator safeguard still applies. This document does not replace those controls; it extends them into the DAG sublayer and resolves the question the earlier baseline could not answer on its own: what exactly prevents the DAG from becoming a new theft, censorship, consistency, or recovery surface?

Baseline Security Implementation Specification v1.0	Remains authoritative for non-DAG attack categories and for cross-cutting wallet, approval, nonce, signature, recovery, and policy controls.
DAG Architecture & Security Specification v1.0	Defines the target architecture. This document translates that design into explicit security obligations, enforcement points, and verification gates.
RPC Specification Part 3	Defines the DAG method surface. This document defines which of those methods may be public, which must be cluster-private, and which must remain operator- or audit-profile only.
PoS consensus materials	Remain authoritative for finality, cluster rotation, and quorum-certificate safety. The DAG must compose beneath those assumptions rather than replace them.

DAG Security Boundary and Non-Negotiable Invariants

These invariants are normative. A validator or operator workflow that violates any one of them is non-compliant even if the network appears to keep running. Synergy is explicitly a safety-over-liveness design, and the DAG layer must inherit that posture.

Invariant	Why it exists	Required enforcement	Violation response
Finality separation	Prevents the DAG from becoming an alternate or ambiguous settlement path.	QC remains the only settlement artifact; DAG certificates only prove availability.	Reject any design or code path that treats a DAG certificate as finality.
Cluster-local scope	Limits flood radius and keeps the attack surface aligned to Synergy cluster boundaries.	Raw vertices, receipts, and repair traffic remain intra-cluster except for compact checkpoints and evidence exports.	Drop or quarantine out-of-scope DAG traffic.
Availability before ordering	Stops unavailable data from influencing canonical state.	No vertex may affect an OrderingCut unless it is availability-certified.	Reject the cut and quarantine the proposer if violated.
Deterministic cut	Prevents local heuristic divergence and hidden ordering games.	Every validator recomputes the same cut from the same certified frontier.	Vote refusal and slashable evidence on mismatch.
One-author-one-vertex-per-round (beta)	Makes equivocation trivial to detect in the first release.	Author, round, epoch, and previous-author link are mandatory in every vertex.	Immediate evidence bundle and penalty pipeline.
Hard epoch boundary	Stops replay and stale graph bleed across validator rotations.	Previous-epoch material expires and cannot be silently imported into the next epoch.	Reject, quarantine, and record replay evidence.
Timestamp is advisory only	Removes wall-clock manipulation from core ordering logic.	Timestamps are bounded metadata; they are never the primary tie-breaker.	Ignore skewed timestamps and flag persistent offenders.
Separate DAG store	Protects recovery, pruning discipline, and forensic retention.	DAG objects, blobs, receipts, and evidence are stored separately from canonical chain state.	Freeze pruning and require archive review if storage invariants fail.
Retention before pruning	Prevents destruction of slashable or reconstructive evidence.	Pruning requires finalised checkpoint lineage, retention expiry, and archive acknowledgement.	Abort pruning and emit operator alert.

Invariant	Why it exists	Required enforcement	Violation response
Fail-closed quarantine	Safety matters more than optimistic throughput.	Validators enter quarantine or safe fallback if DAG consistency cannot be proved.	Stop DAG-influenced ordering rather than continue under uncertainty.

Attack Category 16: Cluster-Local DAG Dissemination and Availability Attacks

16.1 Threat Description

The first DAG-specific danger is believing data exists when it does not. That danger appears in several forms: a validator issues a receipt before it has actually persisted a batch; an author broadcasts a vertex but withholds the encrypted batch blob from a subset of peers; a clique of validators acknowledges only one another and starves the broader frontier; or an attacker saturates peers with low-value vertices, missing-parent references, and fetch pressure until honest nodes cannot maintain a coherent certified frontier.

Synergy must also assume adversaries will try to manipulate parent selection and frontier shape. In a cluster-local setting they show up as clique parents, omission of broadly available parents, selective relay, and deliberately fragmented frontiers that bias ordering opportunities or make honest nodes disagree about which vertices are safe to include.

A batch-withholding attacker wants honest validators to waste memory, fetch bandwidth, and certificate slots on objects that can never become safely orderable. A spam attacker wants the admission queue itself to become the vulnerability. The countermeasure is a stack: strict receipt semantics, per-peer budgets, author discipline, parent-diversity rules, authenticated cluster scoping, and encryption-backed blob commitments.

16.2 Synergy Implementation: Mandatory Availability Certification, Bounded Gossip, and Cluster-Scoped Transport

A validator may issue a receipt only after it has fetched the full encrypted batch or blob reference set, verified the commitment, written the artifact durably to the local DAG store, and confirmed that the blob is retrievable for the retention window. Receipt issuance before durable possession is prohibited. So is speculative receipt issuance based on trust or cached author reputation.

DagCertificate formation is count-thresholded in beta. The natural first-release rule for a cluster of seven is five distinct receipts, i.e. $2f+1$. Synergy Score remains in the finality path, not in the existence path. A vertex exists for ordering only when availability certification proves that enough distinct validators can actually retrieve the data.

Transport is cluster-scoped and authenticated. DAG message families are admitted only from peers that have completed the validator handshake, expose the correct capability flags, and map to the same active cluster membership for the current epoch. Peer budgets are separated by message family so that vertex floods cannot starve QC, checkpoint, or evidence channels. Parent selection must also satisfy a diversity floor.

Encrypted batches are required unless governance explicitly places the network into a controlled fallback profile. The DAG must not reopen the very MEV and pre-trade leakage surface the base Synergy design intends to close. Public RPC surfaces never expose raw batch payloads.

16.3 Implementation Steps

Step	Action	Implementation Detail	Verification Gate
1	Define hard DAG object schemas	Vertex, receipt, certificate, batch-blob reference, and missing-parent request objects must all bind epoch, cluster, round, author identity, sequence number, and typed commitments.	Schema test vectors must round-trip identically across independent implementations.
2	Enforce durable receipt preconditions	Receipt issuance path must verify fetch, checksum, durable write, and local retrieval before signing.	Any receipt emitted before durable persistence is treated as a critical protocol bug.
3	Implement 2f+1 certification	Certificate aggregation must reject duplicate signers, stale epochs, wrong-cluster signers, and invalid bitmaps.	Malformed signer sets must fail certificate verification on every node.
4	Add parent-diversity rules	Beta policy should require parents from distinct authors where available and should reject clique-only parent sets absent a documented network-failure exception.	Frontier simulation must prove that clique references are detected and surfaced.
5	Separate peer budgets by DAG family	Vertices, receipts, certificates, fetches, and evidence each receive their own byte and queue budgets.	Flooding one family must not starve QC, status, or checkpoint traffic.
6	Cluster-authenticate DAG transport	Handshake and capability negotiation must carry validator identity and active cluster eligibility for the epoch.	Cross-cluster raw DAG traffic must be dropped or downgraded to evidence.
7	Bind encrypted batch storage to DAG store	Blob references, checksums, retention timers, and retrieval counters must live in a DAG-specific store separate from chain state.	Recovery testing must reconstruct the frontier without mutating canonical chain state.
8	Wire penalties and quarantine	False receipt issuance, chronic withholding, or spam abuse must trigger Synergy Score penalties, validator status changes, and evidence export.	End-to-end penalty flow must complete without operator hand-editing.

16.4 Verification Tests

TEST SUITE 16A — False Receipt Refusal: Force a validator to attempt receipt issuance before durable batch persistence. The node must refuse to sign, and no certificate assembler may count that receipt even if it appears syntactically valid.
TEST SUITE 16B — Withholding After Vertex Broadcast: Broadcast valid vertices while selectively withholding blobs from a subset of peers. Honest validators must fail certification, surface the withholding signal, and preserve evidence for penalty review.
TEST SUITE 16C — DAG Flood Saturation: Inject low-value vertices, missing-parent requests, and oversized fetch pressure until peer budgets are stressed. QC, checkpoint, and evidence traffic must remain serviceable under independent queue budgets.
TEST SUITE 16D — Parent-Diversity Evasion: Construct a clique that references only itself despite broader parent availability. Honest validators must detect the reduced diversity score and reject or quarantine the offending frontier segment.
TEST SUITE 16E — Subgraph Isolation Under Partial Synchrony: Partition the cluster so different honest subsets see different near-frontiers. The network must avoid certifying unavailable data and must converge once connectivity returns without silent data loss.
TEST SUITE 16F — Public Data Leakage Audit: Confirm that public read RPC endpoints expose status and proofs only, never raw encrypted payloads, plaintext transaction contents, or private repair queues.

Attack Category 17: DAG Ordering, Equivocation, Replay, and Cross-Layer Consistency Attacks

17.1 Threat Description

Once a DAG is certified, the next danger is ordering. A malicious author may equivocate by producing multiple vertices in the same round. A proposer may compute a biased or fraudulent OrderingCut that omits inconvenient certified vertices, includes uncertified vertices, or applies a non-deterministic tie-breaker. An attacker may replay stale vertices into a new epoch, exploit timestamp skew to influence local ordering heuristics, or present a block whose ordered transaction root does not match the certified frontier it claims to consume.

The special risk in Synergy is cross-layer inconsistency. The base layer finalizes blocks with PoSy dual-quorum QCs, while the DAG provides the pre-ordering data plane. If the boundary between those planes is loose, then one validator may believe it voted on a block backed by one frontier while another reconstructs a different frontier from the same commitments. That is exactly the kind of disagreement a deterministic-finality network must refuse to tolerate.

17.2 Synergy Implementation: Canonical Cut Rules, Author Uniqueness, and Block Commitment Binding

The beta rule is intentionally severe: one author, one vertex, one round. Every vertex carries `epoch_id`, `cluster_id`, `round`, `author_validator_id`, `prev_author_vertex_id`, `parent set`, `batch commitment`, `sequence number`, and `author signature`. Same-round equivocation is machine-detectable and slashable.

Ordering is deterministic and implementation-independent. The normative rule for the beta path is: first preserve topological precedence; then sort same-level candidates by round; then by author validator identifier; then by vertex hash; and finally preserve the deterministic transaction order committed within each vertex. Timestamps may be checked for sanity windows and replay diagnostics, but they may not serve as a primary tie-breaker.

Every block that consumes DAG input must bind the DAG evidence it relies on. At minimum the block proposal or header must commit to the OrderingCut root, the included certified-vertex set root, and the availability-related root needed for independent validator reconstruction. Validators must recompute these roots locally before voting.

17.3 Implementation Steps

Step	Action	Implementation Detail	Verification Gate
1	Enforce author uniqueness	Validator runtime must reject or evidence multiple vertices from the same author and round, including variants with different parent sets.	Same-round equivocation must be detected deterministically from signed artifacts alone.
2	Scope all DAG signatures by epoch and cluster	Vertex, receipt, and certificate signatures must bind current epoch and cluster identifiers as first-class fields.	Cross-epoch or cross-cluster replays must fail admission before storage.
3	Implement canonical OrderingCut logic	The cut algorithm and tie-breakers must be identical across validators and covered by fixed test vectors.	Independent implementations must derive the same cut bit-for-bit from the same frontier.
4	Treat timestamps as metadata only	Timestamp checks are bounded-sanity checks and replay diagnostics, never ordering determinants.	Clock skew must not change the resulting cut on honest nodes.
5	Bind DAG roots into block commitments	Block proposals must commit to cut root, certified-set root, and availability root or equivalent roots defined by the DAG architecture spec.	Validators must reject any proposal whose commitments do not recompute locally.
6	Recompute before voting	Validator vote logic must verify certificate validity, frontier eligibility, root agreement, and ordered transaction root agreement before execution voting.	No QC vote may be produced for an unverified DAG-derived proposal.
7	Expire stale graph material at epoch boundaries	Uncertified previous-epoch material must not bleed into the next epoch except through explicit resubmission.	Epoch transition tests must show zero silent carry-over.
8	Emit slashable evidence bundles	Equivocation, fraudulent cuts, root mismatches, and replay attempts must serialize into durable evidence objects consumable by slashing and audit pipelines.	Evidence must survive restart, recovery, and archival export.

17.4 Verification Tests

TEST SUITE 17A — Same-Round Equivocation: Produce two signed same-round vertices from one author. Honest validators must emit the same evidence bundle and refuse both or the offending author according to policy.
TEST SUITE 17B — Deterministic Cut Convergence: Feed identical certified frontiers to multiple independent validator builds and confirm that every build derives the same cut root and ordered transaction root.
TEST SUITE 17C — Fraudulent Proposer Cut: Have a proposer omit certified vertices or inject uncertified vertices into the candidate cut. Validators must reject the proposal before QC voting and preserve evidence.
TEST SUITE 17D — Epoch Bleed and Replay: Reinject previous-epoch vertices, receipts, and certificates into the next epoch. Admission must fail, caches must not silently resurrect them, and replay evidence must be retained.
TEST SUITE 17E — Timestamp Skew: Run validators under skewed local clocks within the allowed bound and confirm that cut computation remains unchanged. Only diagnostics should differ, never ordering.
TEST SUITE 17F — Root Mismatch Audit: Construct a block whose ordered transaction root does not match the cut root or certified-set root it claims to bind. Every honest validator must reject the vote path.

Attack Category 18: DAG Pruning, Recovery, Forensics, and Control-Plane Abuse

18.1 Threat Description

A DAG rollout does not end when the block is finalised. The dangerous long-tail problems live in retention, pruning, recovery, and operator interfaces. If DAG objects are pruned too early, slashable evidence disappears. If archive nodes and validators reconstruct different checkpoint lineages, the network loses forensic credibility. If a privileged operator can trigger repair, replay, or prune procedures without strong audit trails and role isolation, the control plane becomes its own attack surface.

Recovery is especially dangerous because it is where a compromised or merely sloppy node may try to make history fit a desired local state. A malicious restore path can import stale certificates, skip missing evidence, accept inconsistent checkpoints, or rebuild a frontier that no honest validator ever saw. Likewise, a public or weakly protected RPC surface can leak internal DAG topology, private batch handles, or operator-only controls that materially help an adversary map or manipulate the cluster.

18.2 Synergy Implementation: Separate Stores, Proof-Backed Recovery, Restricted DAG RPC, and Quarantine/Fallback

The DAG store is separate from canonical chain state — a hard requirement. Vertices, receipts, certificates, blobs, repair logs, checkpoints, and evidence bundles must be stored in a DAG-specific persistence layer with its own retention rules, integrity digests, and export interfaces. Pruning may occur only below a finalised checkpoint watermark and only after the retention period for slashable evidence and archive reconstruction has expired. If archive acknowledgement or integrity proofs are missing, pruning is blocked.

Recovery must be proof-backed. A node rebuilding DAG state may not simply trust the latest peer that answers first. It must verify checkpoint lineage, certificate validity, signer-set integrity, and stored commitment roots. Where recovery cannot prove a clean DAG state, the node enters quarantine and requests archive or audit assistance.

DAG RPC methods are divided into public read, validator-private, operator, governance-controlled, and audit/forensic profiles. Public read methods expose only safe summaries and proof references. Parameter changes that affect DAG safety — certificate thresholds, parent-diversity rules, retention periods, prune watermarks, safe-fallback toggles — must be governance-controlled and activation-delayed.

18.3 Implementation Steps

Step	Action	Implementation Detail	Verification Gate
1	Separate DAG persistence from chain state	Implement dedicated stores for vertices, blobs, certificates, checkpoints, repair logs, and evidence with independent integrity digests.	Rollback and recovery tests must preserve evidence even when chain replay is restarted.
2	Gate pruning behind finalised lineage and retention	Pruning requires finalised checkpoint watermark, expired evidence-retention timer, and archive acknowledgement or proof of export.	Any missing prerequisite must block pruning automatically.
3	Make recovery proof-backed	Restore logic must verify certificate signer sets, checkpoint lineage, object hashes, and epoch scope before admitting recovered data.	Recovery from a corrupted peer must fail closed rather than silently heal to a bad state.
4	Define DAG RPC exposure classes	Each method family must declare whether it is public read, validator-private, operator, or audit-only and must enforce that class in transport and auth policy.	Public deployment scans must show zero operator or validator-private DAG methods exposed on public profiles.
5	Audit every privileged DAG action	Prune, replay, quarantine release, fallback activation, and private repair operations must emit immutable operator audit events.	Operator log tampering attempts must be detectable and reviewable.
6	Implement quarantine and safe fallback	Nodes with unproved DAG state must stop DAG-influenced ordering and either await recovery or revert to the narrowly defined safe fallback profile.	Fallback activation must never bypass QC or invent unproved DAG data.
7	Protect public observability surfaces	Metrics and status endpoints must aggregate and redact enough detail that they support operations without leaking private batch or repair data.	Security review must confirm that status endpoints do not materially aid target selection.
8	Delay and sign DAG safety-parameter changes	Governance and operator tooling must treat threshold, pruning, retention, and exposure changes as signed, delayed activations.	No immediate unsafe parameter flip may take effect on a live cluster.

18.4 Verification Tests

TEST SUITE 18A — Premature Pruning Block: Attempt to prune certified DAG material before retention expiry or before archive acknowledgement. The node must block the operation and emit an operator audit event.
TEST SUITE 18B — Corrupted Recovery Source: Restore from a peer that serves valid-looking but inconsistent checkpoints or signer sets. The node must quarantine rather than merge the bad state into local operation.
TEST SUITE 18C — Public RPC Exposure Scan: Enumerate every DAG endpoint from a public profile. Only safe summary and proof-reference methods may be reachable; validator-private, operator, and audit methods must remain inaccessible.
TEST SUITE 18D — Audit Trail Integrity: Perform prune, replay, quarantine release, and safe-fallback activation actions. Every action must create immutable operator audit records with actor identity and timestamp.
TEST SUITE 18E — Evidence Retention Through Restart: Create slashable DAG evidence, restart the node, and confirm that evidence remains locally retrievable and exportable to audit roles.
TEST SUITE 18F — Unsafe Parameter Change Delay: Attempt to lower certificate thresholds or shorten evidence retention immediately. The activation must be delayed, authorized, and vetoed if it violates mandatory invariants.

Attack Category 19: CRQC On-Spend Attack on Active Transactions

19.1 Threat Description

A fast-clock CRQC (superconducting, photonic, or silicon-spin architecture) can break 256-bit ECDLP in approximately 9 minutes per Paper 2's verified resource estimates (1,200 logical qubits, 90M Toffoli gates — verified via published zero-knowledge proof). This creates a category of attack where a CRQC operator observes a classical transaction in a public mempool, derives the signing private key before the transaction is finalized, and injects a forged transaction redirecting the funds.

On-spend attack feasibility by chain block time:

Chain	Avg Block Time	On-Spend Risk (9-min fast-clock CRQC)
Bitcoin	~10 minutes	~41% success probability (Paper 2, Figure 6). HIGH RISK.
Ethereum	12 seconds	Extremely low — settled before key can be broken.
Solana	400 milliseconds	Zero — settled orders of magnitude faster than CRQC break time.
Synergy (target)	~5 seconds (PoSy BFT)	Zero — BFT finality within seconds. Two independent defenses: (1) PoSy BFT finalizes before any known CRQC completes key derivation; (2) encrypted mempool prevents public key observation entirely.

Synergy is structurally resistant to on-spend attacks for two independent reasons: (1) PoSy's deterministic BFT finality produces irreversible blocks in seconds, before any known CRQC architecture could complete the key derivation; and (2) the encrypted mempool prevents an attacker from observing transaction content to identify target public keys. These defenses must be explicitly documented and verified.

19.2 Synergy Implementation

Defense is provided by the combination of PoSy BFT finality speed and the encrypted mempool threshold decryption system. Neither defense alone is sufficient in isolation — both must be maintained and tested together.

19.3 Implementation Steps

1. DOCUMENT encrypted mempool's role as an on-spend attack defense — not just an MEV/sandwich defense. Update Category 8 documentation to explicitly note the dual function. (See Category 8 addition above.)
2. VERIFY that PoSy BFT finality is achieved before any known CRQC architecture (including future fast-clock systems) could complete a private key derivation. Document the safety margin explicitly in the validator specification.
3. DEFINE a Synergy-specific attack surface assessment: at current target block time, what CRQC clock speed would be required to mount an on-spend attack? Establish and document this threshold as a protocol safety bound.
4. MONITOR quantum hardware clock speed advances. Establish a governance trigger: if superconducting qubits achieve <1ms stabilizer cycles (potentially enabling sub-minute ECC-256 breaks), Synergy's encrypted mempool + BFT speed combination must undergo hardening review.

19.4 Verification Test Suites

TEST SUITE 19A — Mempool Key Exposure Audit

Simulate a mempool observer node and attempt to recover transaction content before the decryption threshold is reached. Confirm zero information leakage about signing public keys for any transaction type, including SXCP cross-chain relay transactions.

TEST SUITE 19B — BFT Finality Adversarial Timing

Measure empirical PoSy BFT finality time under adversarial network conditions (30% Byzantine validators, high latency, packet loss simulation). Confirm finality is achieved in <30 seconds under worst-case conditions, preserving a substantial safety margin against even optimistic CRQC clock speed assumptions.

Attack Category 20: CRQC At-Rest Attack on Long-Lived Exposed Keys

20.1 Threat Description

Paper 1 establishes that a neutral-atom CRQC with ~26,000 physical qubits can break ECC-256 in approximately 10 days. A CRQC operator can target exposed public keys in order of economic value, deriving private keys for the highest-value targets first. This is an at-rest attack: the target key does not need to be in use, it only needs its public key to have been previously exposed on-chain.

The relevant at-rest surfaces on Synergy are not Synergy's own keys (which use ML-DSA and are Shor-resistant), but the cross-chain ECDSA keys that Synergy validators, relayers, and cross-chain adapters must interact with:

- SXCP relayer keys on external EVM chains: If a Synergy-side SXCP relayer maintains an ECDSA wallet on Ethereum or another classical chain, and that key has been used (public key exposed on-chain), it is a permanent at-rest target.
- Chain binding verification keys: The cross-chain identity layer's EVM adapters require users to prove ownership of ECDSA addresses. Submitted proofs include ECDSA public keys, which are permanently stored on-chain.
- SXCP WitnessRegistry on external chains: The WitnessRegistry smart contract stores relayer public keys. If deployed on an ECDSA-based chain, these are permanently exposed at-rest targets.
- User wallets interacting with Synergy cross-chain: Users bridging assets from Ethereum to Synergy expose their Ethereum ECDSA public key. A CRQC attacker can derive the private key and drain their Ethereum holdings even after those users have migrated to Synergy's ML-DSA identity.

20.2 Synergy Implementation

Synergy's native protocol is immune to at-rest attacks on its own keys due to ML-DSA. The implementation requirement is to map, manage, and migrate the cross-chain ECDSA exposure surface.

20.3 Implementation Steps

5. DOCUMENT a 'Cross-Chain Key Exposure Registry' — for every chain Synergy interacts with via SXCP, identify which classical public keys are stored on-chain and what assets or authorities they control.
6. DESIGN a SXCP 'quantum firewall' pattern: for any cross-chain interaction that requires proving ownership of a classical ECDSA address, Synergy should complete the attestation and immediately propagate a signed Synergy-native (ML-DSA) binding, reducing ongoing dependence on the exposed classical key.
7. REQUIRE SXCP relayer operators to use separate 'operational ECDSA keys' with minimal asset holdings for chain interactions, and 'cold governance ML-DSA keys' for all significant operations. Document this as a mandatory validator operational requirement.
8. PUBLISH a PQC migration recommendation to all Synergy users interacting cross-chain: any Ethereum/Solana/Aptos/Sui address bound to a Synergy DID that has been used (public key on-chain) should be migrated to a new address before the first CRQC is operational.
9. ESTABLISH a Synergy oracle service that monitors quantum computing hardware milestones and issues on-chain alerts when the at-rest attack threshold approaches (e.g., when neutral atom qubit count reaches 5,000+ in a fault-tolerant configuration).

20.4 Verification Test Suites

TEST SUITE 20A — Cross-Chain Public Key Exposure Audit

For each supported cross-chain (EVM, Solana, Aptos, Sui, Starknet), enumerate all classical public keys stored on-chain by Synergy's smart contracts, SXCP contracts, or binding proofs. Confirm each has a documented migration plan with owner, timeline, and fallback procedure.

TEST SUITE 20B — At-Rest Attack Scale Projection

Simulate an at-rest attack scenario: given a CRQC that can derive private keys at 14 hours per key (slow-clock, 10K qubit model from Paper 2), how many Synergy-adjacent ECDSA keys would be derivable in 30 days? Confirm the count is within acceptable risk tolerance and that high-value keys have migration plans that precede the 2–4 year CRQC threshold.

Attack Category 21: CRQC Quantum Consensus Attack via Validator Key Compromise

21.1 Threat Description

Paper 2 (Section V.D) analyzes a class of attack specific to proof-of-stake consensus: a CRQC operator systematically derives private keys for validator consensus signing keys, eventually compromising enough stake to halt finality ($>1/3$ stake) or rewrite history ($>2/3$ stake). This attack is generically applicable to any PoS mechanism using quantum-vulnerable signing keys.

Ethereum's PoS consensus uses BLS12-381 signatures, which are quantum-vulnerable. Synergy's PoSy is structurally immune to this attack because it uses ML-DSA-65 for all validator signing — a Shor-resistant lattice-based signature scheme. This immunity must be formally documented. The residual risk is cross-chain governance: classical admin keys on external chains that could be used to manipulate the validator set via governance votes must be identified and managed.

21.2 Synergy Implementation

PoSy consensus signing uses ML-DSA-65 exclusively. No BLS, ECDSA, or other quantum-vulnerable signature scheme is used in the PoSy consensus path. This is a design invariant that must be enforced via protocol rule and SynQ compiler constraint.

21.3 Implementation Steps

10. ADD a 'Quantum Consensus Safety Proof' to the spec: explicitly document that PoSy validator consensus uses ML-DSA-65, that ML-DSA-65 is Shor-resistant per FIPS 204, and that the consensus layer therefore has no CRQC attack surface on its signing keys.
11. DOCUMENT the SXCP relayer quorum separately: SXCP's WitnessRegistry and threshold attestation must use ML-DSA keys. Confirm and document that no ECDSA keys are used in the SXCP threshold signing path.
12. ESTABLISH a protocol invariant: no governance action may introduce classical (non-ML-DSA) signing keys into the consensus or SXCP signing path. Enforce this invariant via SynQ compiler constraints that reject any signing operation using non-ML-DSA key types.
13. ADD to governance spec: any protocol upgrade that would introduce classical key material into consensus signing requires a supermajority vote plus a mandatory quantum security review by the designated Quantum Threat Tracking authority.

21.4 Verification Test Suites

TEST SUITE 21A — Consensus Signing Key Audit

Enumerate all active validator consensus signing keys across a test cluster. Confirm every key is ML-DSA-65. Confirm no legacy key material of any other type is present in the validator runtime.

TEST SUITE 21B — Governance Upgrade Constraint Test

Attempt to submit a governance proposal that would introduce an ECDSA key into the consensus signing path. Confirm the SynQ compiler and protocol runtime reject the proposal before it can be voted on or executed.

 **NEW — Attack Category 22: On-Setup Quantum Attack on ZK Trusted Setups**

Attack Category 22: On-Setup Quantum Attack on ZK Trusted Setups

22.1 Threat Description

Paper 2 introduces and analyzes an attack type with a unique property: it requires a CRQC to execute only once, but the resulting exploit is classical and reusable indefinitely. Some ZK proof systems (KZG polynomial commitments, Groth16 SNARK setup, Tornado Cash) require a one-time trusted setup ceremony producing public parameters. These parameters encode a secret scalar that must be destroyed. A CRQC can recover this scalar via ECDLP on the public parameters — a single operation — and the resulting exploit allows arbitrary proof forgery on a classical computer forever after.

Synergy's attack surface for this category:

- Synergy's own ZK layer: Uses STARK-based proofs (hash-based). STARKs have no trusted setup and are quantum-resistant. Synergy itself is NOT vulnerable to on-setup attacks.
- Cross-chain SXCP risk: Synergy's SXCP protocol interacts with Ethereum's L2 ecosystem, which as of 2026 uses KZG commitments for Data Availability Sampling (EIP-4844/7594). An on-setup attack on Ethereum's KZG SRS would compromise Ethereum L2 data availability, affecting any SXCP-mediated transfers to/from Ethereum L2s.
- Future component risk: If any Synergy component ever uses Groth16-based SNARK proofs (e.g., for EVM compatibility layer verification), those components would inherit this vulnerability. The spec must explicitly prohibit Groth16 usage.

22.2 Synergy Implementation

STARK-based ZK proofs are mandatory for all Synergy protocol components. Groth16 and all pairing-based SNARK schemes are prohibited by protocol rule and SynQ compiler enforcement.

22.3 Implementation Steps

- 14. ADD an explicit protocol rule: SynQ contracts and Synergy protocol components are prohibited from using Groth16 (BN254 / BLS12-381 pairing-based) SNARK proofs. Only STARK-based or hash-based ZK proofs are permitted. Enforce this via SynQ compiler flag that rejects pairing-based proof generation calls.
- 15. DOCUMENT the cross-chain on-setup risk: create a threat model entry for SXCP interactions with Ethereum L2s that use KZG. Identify which SXCP operations depend on Ethereum DAS availability and what happens if DAS is compromised by a KZG on-setup attack.
- 16. DEFINE a contingency mode for SXCP: if an Ethereum KZG on-setup attack is detected (via Synergy's SXCP witnesses), define the protocol's response — including whether SXCP cross-chain transfers to/from affected Ethereum L2s should be paused pending assessment.
- 17. ADD to ZKProofVerifier contract specification: the verifier must explicitly reject any proof using pairing-based curve operations (alt_bn128, BLS12-381). Only STARK-compatible proof formats may be accepted.

22.4 Verification Test Suites

TEST SUITE 22A — ZK Proof Format Enforcement

Submit a Groth16-formatted proof to the ZKProofVerifier contract. Confirm rejection. Submit a proof using BLS12-381 pairing operations. Confirm rejection. Submit a valid STARK proof. Confirm acceptance.

TEST SUITE 22B — SynQ Compiler Pairing Prohibition

Attempt to compile a SynQ contract that imports or calls pairing-based proof generation. Confirm the compiler rejects the contract at compile time with an explicit error referencing the Groth16 prohibition rule.

 **NEW — Attack Category 23: Harvest-Now-Decrypt-Later (HNDL) Cross-Chain Attack**

Attack Category 23: Harvest-Now-Decrypt-Later (HNDL) Cross-Chain Attack

23.1 Threat Description

HNDL is a class of attack that does not require breaking cryptography at the time of collection. An adversary records classical cryptographic artifacts today — ECDSA signatures, ECDH ephemeral secrets, SXCP relayer attestations on classical chains — and processes them through Shor's algorithm once a CRQC becomes available. The attacker does not need to break the cryptography now; they only need to store what is publicly observable on-chain.

Critically, the two milestone rows in the Quantum Threat Timeline above (6,100 qubits demonstrated, fault-tolerant operation below threshold) establish that HNDL mode is not a future threat — it is the current threat. Every classical signature recorded on-chain today is a potential HNDL artifact.

Synergy's HNDL surface is not in the core protocol (which uses ML-DSA throughout), but in the cross-chain interaction layer:

- Every EVM chain binding proof submitted to Synergy's ChainBindingVerifier contains an ECDSA signature. These are stored on-chain permanently and can be harvested now and decrypted later to derive the signing key.
- SXCP attestations on Ethereum/Solana/Aptos/Sui that use those chains' native signing schemes (secp256k1, Ed25519) are HNDL targets. Once harvested and later broken, an attacker can forge retroactive attestations.
- Any Synergy user's Ethereum address that has transacted is already a permanent HNDL target. A CRQC can derive their private key from on-chain signature data and drain their Ethereum holdings — even if they have migrated to Synergy's ML-DSA identity.
- KZG/Groth16 proof artifacts submitted to any verifier that Synergy interacts with are HNDL targets for on-setup attacks (see Category 22).

23.2 Synergy Implementation

HNDL risk cannot be eliminated retroactively for data already on classical chains. The implementation strategy is (1) document the full surface, (2) implement expiry and versioning controls on new classical-key binding proofs, and (3) provide migration tooling and guidance to reduce the surface over time.

23.3 Implementation Steps

18. DOCUMENT the HNDL surface: for every cross-chain signing operation Synergy participates in or verifies, identify which are HNDL targets and what the harvested data enables a future CRQC operator to do.
19. IMPLEMENT binding proof expiry: ChainBindingVerifier must enforce that classical ECDSA binding proofs have an expiry block. After expiry, the proof must be refreshed. Once PQC wallets are available on the target chain, users must be migrated to ML-DSA binding proofs.
20. PUBLISH a HNDL migration guide to Synergy developers and users: explain that any EVM/Solana address whose public key is on-chain is a HNDL target, describe the threat model precisely, and provide the migration path to PQC addresses on those chains.
21. DEFINE a binding proof version field in ChainBindingVerifier that distinguishes classical-key bindings (HNDL-vulnerable) from PQC bindings (HNDL-resistant). This field enables tracking migration progress and eventually deprecating classical bindings on a governance-controlled schedule.

23.4 Verification Test Suites

TEST SUITE 23A — HNDL Surface Audit

Enumerate all classical-key artifacts stored on-chain by Synergy contracts across all supported chains. For each artifact type, document: the signing scheme, the HNDL window (how long it will be stored), and the migration path. Confirm all artifact types have associated expiry or versioning controls.

TEST SUITE 23B — Binding Proof Expiry Enforcement

Submit a classical ECDSA binding proof to ChainBindingVerifier. Advance the chain past the expiry block. Attempt to use the expired proof. Confirm rejection. Submit a valid ML-DSA binding proof with the correct version field. Confirm acceptance.

DAG Security Control Matrix

This matrix condenses the DAG threat surface into primary enforcement points. It is intended for protocol engineering reviews, validator runbooks, and external auditors who need a clear mapping from failure mode to control stack.

Risk	Primary control	Secondary control	Evidence / response
False availability receipt	Durable fetch-and-persist required before receipt issuance.	Certificate assembler rejects malformed or duplicate signers.	Receipt fraud evidence; penalty and quarantine.
Batch withholding	Availability before ordering; certification threshold.	Repair path and missing-blob diagnostics.	Withholding bundle; author penalty.
Missing-parent poisoning	Explicit fetch flow and bounded retry logic.	Peer budget separation for fetch traffic.	Peer score reduction; evidence if malicious.
Clique parent selection	Parent-diversity minimums in validation.	Frontier health metrics and audit review.	Reject or deprioritize clique-biased vertices.
Frontier flood / spam	Per-peer byte budgets and queue partitioning.	Author rate limits and admission windows.	Automatic throttling and operator alert.
Same-round equivocation	One-author-one-vertex-per-round rule.	Prev-author link and signer identity binding.	Immediate slashable evidence.
Epoch replay / bleed	Epoch-scoped IDs, signatures, and caches.	Hard expiry at epoch transition.	Reject and record replay attempt.
Timestamp manipulation	Timestamp only advisory, not ordering key.	Bounded skew diagnostics.	Ignore for ordering; flag offender.
Fraudulent OrderingCut	Deterministic cut recomputation by every validator.	Block commitment roots for cut and certified set.	Vote refusal and evidence export.
QC / checkpoint mismatch	Cross-layer root binding and lineage verification.	Audit reconstruction from archive roles.	Quarantine until lineage is proved.
Pruning corruption	Retention gate + finalised watermark + archive ack.	Separate DAG store and integrity digests.	Prune blocked; forensic review.
Archive divergence	Proof-backed recovery and lineage checks.	Audit validator cross-check.	Quarantine bad lineage; do not merge.
Public RPC information leakage	Profile-based exposure control.	Redacted summaries only on public endpoints.	Security review and deployment hardening.
Operator abuse	mTLS, RBAC, and immutable audit logs.	Delayed activation for safety-sensitive changes.	Action traceability and rollback review.
MEV leakage through DAG	Encrypted batches and non-public blob access.	No raw payload exposure on public methods.	Leak audit and emergency fallback policy.

DAG RPC and Operational Security Rules

The DAG RPC surface is a security boundary, not just a developer surface. The same method can be perfectly reasonable in an authenticated cluster and catastrophically irresponsible on a public endpoint. Synergy must therefore bind every DAG method family to an exposure profile that is enforced in transport, authorization, and deployment policy rather than left as a dashboard toggle.

Public read DAG methods may return node status, safe summaries, proof references, and aggregated metrics. They may not expose private batch handles, raw payloads, active repair queues, or privileged topology hints. Validator-private methods must assume hostile observation and require authenticated cluster membership, capability negotiation, and audit-friendly request traces.

Operator and forensic DAG methods deserve special caution because they can influence safety posture directly. Quarantine release, replay control, prune planning, parameter overrides, and evidence export must be protected by mutually authenticated transport, role-based authorization, immutable audit records, and non-public routing. A convenient operator shortcut that bypasses those controls is not an operational feature; it is a latent incident report.

Method family	Profile	Mandatory controls	Never expose / never permit
Status, health, metrics	Public read	Summary-only responses, rate limits, no private payload handles.	Do not leak raw blobs, repair queues, or peer secrets.
Object/proof retrieval	Public read or audit depending on payload	Proof references only on public profiles; full evidence only on audit profiles.	No public plaintext batch retrieval.
Vertex/receipt/certificate submission	Validator-private	Cluster membership auth, capability check, per-peer quotas, audit trace.	Never expose on public internet endpoints.
Repair, replay, recovery assist	Operator or audit	mTLS, RBAC, immutable audit log, bounded scopes.	No anonymous recovery control.
Quarantine, fallback, prune planning	Operator	Two-person review where available, delayed activation, audit logging.	No immediate unsafe override outside policy.
Evidence export and forensic lineage	Audit / authorized operator	Integrity verification, export logging, retention-aware access.	Never permit silent evidence deletion or overwrite.

Red-Team and Verification Matrix

The following scenarios are the minimum red-team set for a serious DAG rollout. Passing them does not prove perfection, but failing them means the network is not ready for security-sensitive deployment.

Scenario	Adversary objective	Pass criterion
Selective blob withholding	Get a vertex ordered without cluster-wide retrievability.	No unavailable vertex may influence the cut or final block.
Receipt fraud	Mint certification from nodes that never stored data.	Receipt issuance path refuses early signatures; bogus receipts do not aggregate.
Cluster clique formation	Bias parent selection and frontier shape.	Diversity checks detect and suppress clique-only reference patterns.
Flood and queue starvation	Starve QC or checkpoint channels with DAG traffic.	Independent queue budgets preserve consensus and evidence traffic.
Same-round equivocation	Create ambiguous author history.	Evidence is deterministic and slash pipeline can consume it automatically.
Cross-epoch replay	Smuggle stale DAG artifacts into new cluster context.	Admission rejects stale artifacts before ordering or storage.
Fraudulent proposer cut	Exploit omitted or uncertified vertices for ordering advantage.	Validators recompute, reject, and preserve proof of fraud.
Checkpoint lineage corruption	Convince recovering nodes to accept bad graph history.	Recovery enters quarantine instead of adopting unproved lineage.
Premature pruning	Destroy slashable evidence or recovery-critical artifacts.	Prune request is blocked until all prerequisites are satisfied.
Public RPC reconnaissance	Map cluster internals or private payload availability.	Public methods reveal only safe summaries and proof references.
Operator-plane misuse	Disable controls or activate unsafe fallback silently.	All sensitive actions require auth, audit, and delayed activation.
Encrypted batch leakage	Recover pre-order transaction content for MEV advantage.	Payloads remain encrypted and inaccessible on public surfaces.

Quantum Threat Timeline and Milestone Tracking

The security specification must establish the current quantum hardware state, project milestone thresholds, and define Synergy's response obligations at each milestone. This section establishes those obligations as normative requirements, not advisory guidance.

Source basis: Cain, Xu, King et al. (arXiv:2603.28627, March 31, 2026) and Babbush, Zalcman, Gidney et al. (Google Quantum AI / Ethereum Foundation, March 30, 2026). Both papers represent the current state of the art in quantum threat assessment and materially compress prior timeline estimates.

Milestone	Current Status (April 2026)	Estimated Timeline	Synergy Response Required
6,000+ coherent atomic qubits demonstrated	ACHIEVED — Manetsch et al., 2025: 6,100 qubits demonstrated	Complete	HNDL awareness mode: all classical key material must be treated as harvestable now. Cross-chain ECDSA public keys on-chain are permanent HNDL targets.
Fault-tolerant neutral atom operation below error threshold	ACHIEVED — Bluvstein et al., 2026: below-threshold demonstrated on 500 qubits	Complete	At-rest attack preparation: validator key audit, cross-chain key exposure mapping, HNDL surface documentation complete.
10,000+ qubit neutral atom system with fault tolerance	NOT YET — engineering gap from 6,100 to 10,000 qubits	2–4 years (Paper 1 estimate)	At-rest attack imminent: PQC migration for all cross-chain ECDSA keys must be complete. SXCP quantum firewall must be active. All classical chain binding proofs must have expiry enforcement.
Fast-clock CRQC at ~500K superconducting qubits	Logical circuits EXIST (Paper 2 ZK proof); physical realization pending	3–6 years	On-spend protection: verify encrypted mempool + BFT finality margin still sufficient. Full PQC cross-chain migration complete. Any remaining classical key interactions frozen.
First confirmed CRQC breaking real blockchain asset	Not achieved	Unknown — could precede public announcement; on-chain detection may be first signal	Emergency response: trigger Quantum Emergency Governance Protocol (see section below). Pause all SXCP cross-chain transfers to/from chains using classical signing.

GOVERNANCE REQUIREMENT:
This timeline must be actively maintained. Synergy engineering must designate a Quantum Threat Tracking responsibility with quarterly review cadence. When a milestone's estimated timeline compresses by more than 12 months, an immediate spec revision is required.

Quantum Emergency Governance Protocol

This section defines Synergy's mandatory response when a CRQC is first confirmed to have been used to steal blockchain assets. Detection may occur on-chain before any public announcement, as anomalous transaction patterns may be identifiable via SXCP witnesses or cross-chain monitoring infrastructure before a public disclosure is made.

Phase	Required Actions
Immediate (0–24 hours)	• Pause all SXCP cross-chain transfers to/from chains using classical signing schemes (ECDSA/secp256k1/Ed25519) • Halt acceptance of new classical-key binding proofs in ChainBindingVerifier • Activate on-chain alert via quantum oracle service (per Category 20, Step 5) • Notify all SXCP relator operators to suspend classical-chain operations pending assessment
Short-term (24–48 hours)	• Submit governance emergency proposal to formally declare quantum emergency state • Activate SXCP contingency mode for affected chains (per Category 22, Step 3)
7-day horizon	Publish migration guidance for all users with classical cross-chain bindings. Provide tooling for emergency migration of high-value ECDSA keys to new addresses.
90-day horizon	Deprecate all classical-key chain bindings in ChainBindingVerifier in favor of PQC-only bindings. Resume SXCP operations with PQC-only attestation paths for all chains that support them.

GOVERNANCE CONSTRAINT:
The Quantum Emergency Governance Protocol is a mandatory response plan, not a discretionary option. The governance body may not override these response obligations. Protocol invariants established in Categories 19–23 remain in force throughout the emergency response period.

Implementation Governance and Deployment Gates

A secure DAG rollout is not a code merge; it is a gated programme. Synergy should not move directly from schema design to production operation. The network needs a tight sequence of decision gates because the dangerous failures are emergent failures — the kind that appear only when consensus, transport, storage, operator tooling, and RPC policy are all live together.

Gate	Required outcome	Hard stop if absent
Gate 1 — Parameter freeze	Canonical decisions on beta cluster size, epoch duration, certificate threshold, parent-diversity floor, and safe-fallback profile.	Do not implement DAG ordering with unresolved parameter drift.
Gate 2 — Schema and test vectors	Finalised object formats, signature domains, and deterministic cut test vectors.	No multi-team implementation without canonical vectors.
Gate 3 — P2P hardening	Cluster-authenticated transport, queue budgets, and message-family rate limits in place.	No load testing on an unauthenticated raw DAG plane.
Gate 4 — Validator verification path	Every validator recomputes DAG roots before QC voting.	No DAG-derived proposal may reach QC on proposer trust alone.
Gate 5 — Recovery and retention	Archive, audit, evidence export, and prune gates implemented and exercised.	Do not enable pruning or long-lived DAG history without proof-backed recovery.
Gate 6 — RPC exposure audit	Public, validator-private, operator, and audit profiles enforced and externally scanned.	Do not publish DAG endpoints without profile verification.

Governance may authorize DAG deployment, but governance may not authorize the validator runtime to violate mandatory invariants. If a requested policy would permit uncertified ordering, premature pruning, public private-batch exposure, or non-deterministic cut construction, the implementation must refuse it. That is not insubordination; it is the point of having a security specification.

Complete v1.3 Quantum Evaluation Finding Summary

ID	Category	Severity	Description	Action in v1.3
A-1	ALIGNED	—	ML-DSA-65/87 as signing algorithm throughout	No change — confirmed correct
A-2	ALIGNED	—	ML-KEM-768 for key exchange	No change — confirmed correct
A-3	ALIGNED	—	SLH-DSA for long-lived credentials	No change — confirmed correct
A-4	ALIGNED	—	No BLS signatures in PoSy	No change — BLS confirmed quantum-vulnerable
A-5	ALIGNED	—	STARK-based ZK proofs, not Groth16	No change — STARKs confirmed quantum-resistant
A-6	ALIGNED	—	Encrypted mempool prevents on-spend key observation	Dual-function documentation added to Category 8
A-7	ALIGNED	—	Key rotation without identity change (UMA)	Documented as explicit Shor defense in Cat. 12 clarification
A-8	ALIGNED	—	No KZG or pairing-based commitments in core Synergy	No change — KZG confirmed vulnerable to on-setup attacks
M-1	MISALIGNED	CRITICAL	Quantum threat framed as future/distant concern	All timeline language updated; quantum threat framed as active engineering constraint throughout
M-2	MISALIGNED	IMPORTANT	No Shor's/Grover's algorithm analysis	Threat decomposition section added
M-3	MISALIGNED	IMPORTANT	No fast-clock vs slow-clock CRQC distinction	CRQC hardware taxonomy section added
M-4	MISALIGNED	MODERATE	PQC migration implies classical key interim use is safe	Category 12 clarification added: PQC from genesis, no safe interim period
G-1	GAP	CRITICAL	Category 19: CRQC on-spend attack missing	Category 19 added in full
G-2	GAP	CRITICAL	Category 20: CRQC at-rest attack on long-lived keys missing	Category 20 added in full
G-3	GAP	HIGH	Category 21: Quantum consensus validator attack missing	Category 21 added in full
G-4	GAP	HIGH	Category 22: On-setup ZK trusted setup attack missing	Category 22 added in full
G-5	GAP	HIGH	Category 23: HNDL cross-chain harvest attack missing	Category 23 added in full
G-6	GAP	HIGH	Quantum threat timeline absent entirely	Quantum Threat Timeline section added; Quantum Emergency Governance Protocol added